Module 5: Flow Networks

Serge Gaspers (K17 506)

Course Admins: cs3821@unsw.edu.au
(Liam, Gerald, Song)

School of Computer Science and Engineering
UNSW Sydney

Term 3, 2025

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

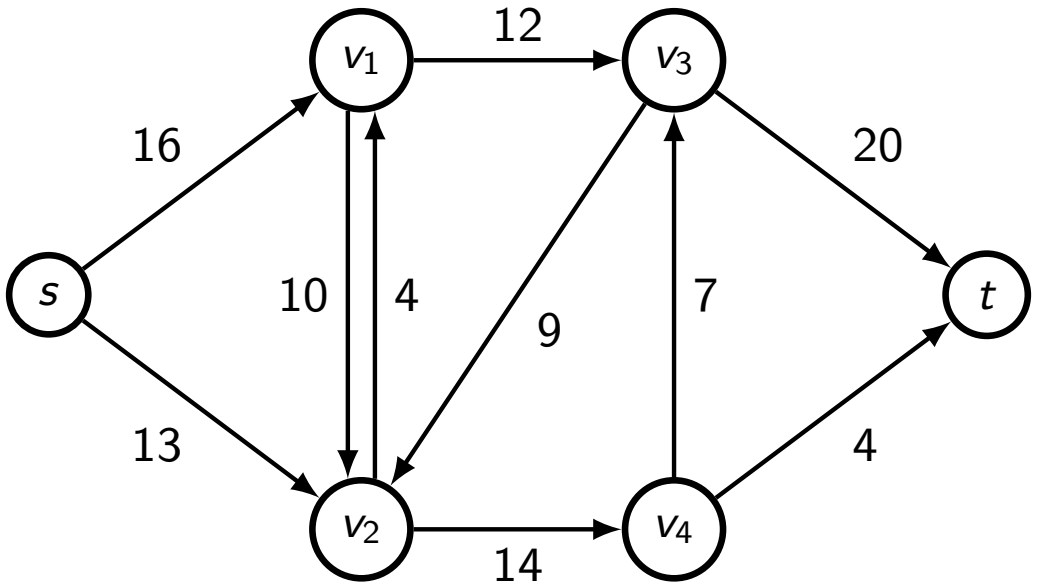
5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Definition

A flow network $G = (V, E)$ is a directed graph in which each edge $e = (u, v) \in E$ has a positive integer capacity $c(u, v) > 0$.
There are two distinguished vertices: a source s and a sink t.
No edge enters the source, and no edge leaves the sink.



Flow Networks

Examples of flow networks (possibly with multiple sources and multiple sinks):

- transportation networks
- gas pipelines
- computer networks
- and many more.

Definition

A flow in G is a function $f : E \to [0, \infty)$, $f(u, v) \geq 0$, which satisfies

1. capacity constraint: for each edge $e = (u, v) \in E$ we have

$$f(u, v) \leq c(u, v),$$

   i.e., the flow through any edge does not exceed its capacity.
2. flow conservation: for each vertex $v \in V \setminus \{s, t\}$ we have

$$\sum_{(u,v) \in E} f(u, v) = \sum_{(v,w) \in E} f(v, w),$$

   i.e., the flow into any vertex (other than the source and the sink) equals the flow out of that vertex.

Definition

The <u>value</u> of a flow is defined as

$$|f| = \sum_{v:(s,v)\in E} f(s,v) = \sum_{v:(v,t)\in E} f(v,t),$$

i.e., the flow leaving the source, or equivalently, the flow arriving at the sink.

Given a flow network, our goal is to find a flow of maximum value.

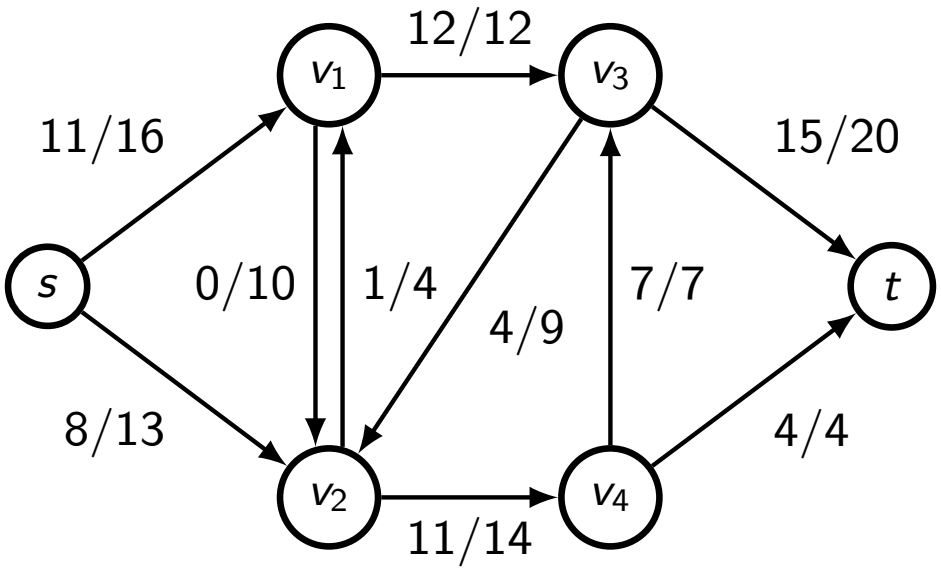
Integrality Theorem

If all capacities are integers (as we assumed earlier), then there is a flow of maximum value such that $f(u, v)$ is an integer for each edge $(u, v) \in E$.

Note

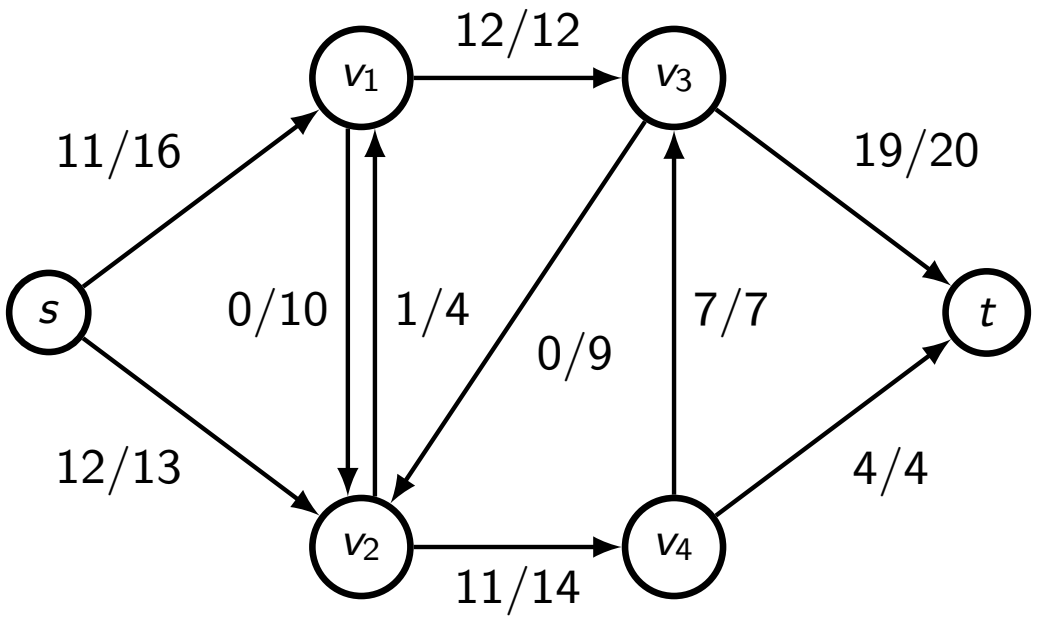
This means that there is always at least one solution entirely in integers. We will only consider integer solutions hereafter.

In the following example, f/c represents f units of flow sent through an edge of capacity c.



Question. Is this flow valid? In other words, does it satisfy the capacity constraints and flow conservation? Yes!
Question. What is the value of the flow? 19 units.
Question. Is this the maximum flow possible? No, although this may not be obvious.

Here is a flow of value 23 units in the same flow network:



Exercise. Check this satisfies the capacity constraints and flow conservation.

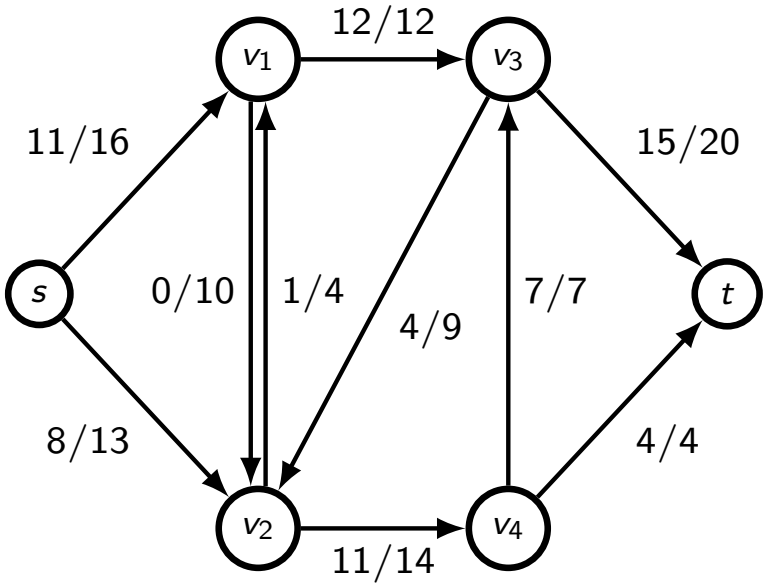
Exercise. Can you find an argument for why this flow is maximum?

- This example demonstrates that the most basic greedy algorithm - send one unit of flow at a time along arbitrarily chosen paths - does not always achieve the maximum flow.
- What went wrong in the first attempt?
- We sent 19 units of flow to vertex v_3, only to send four units back to v_2.
- It would have been better to send those four units of flow to t directly, but this may not have been obvious at the time this decision was made.
- We need a way to correct mistakes! We would like to send flow from v_2 back to v_3 so as to "cancel out" the earlier allocation.

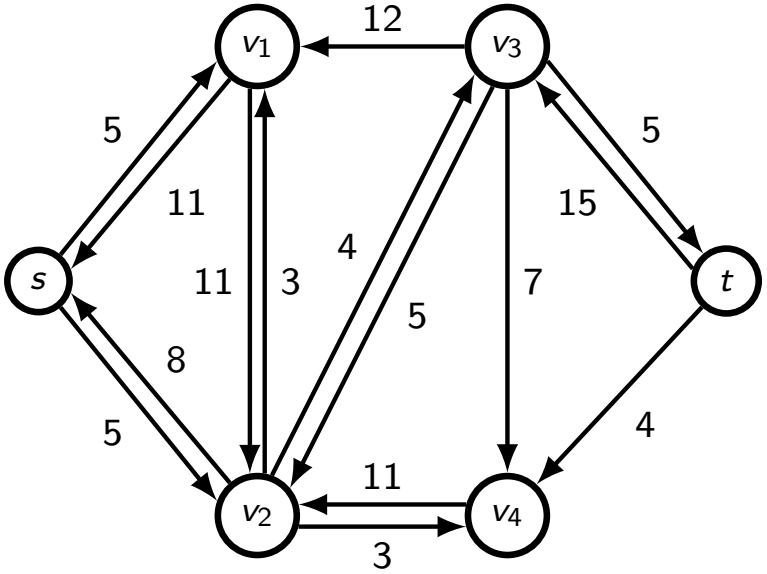
Definition

Given a flow in a flow network, the residual flow network is the network made up of the leftover capacities.

Flow network



Residual flow network



Residual Flow Network

- Suppose the original flow network has an edge from v to w with capacity c, and that f units of flow are being sent through this edge.
- The residual flow network has two edges:
  1. an edge from v to w with capacity $c - f$, and
  2. an edge from w to v with capacity f.
- These capacities represent the amount of additional flow that can be sent in each direction. Note that sending flow on the "virtual" edge from w to v counteracts the already assigned flow from v to w.
- Edges of capacity zero (when $f = 0$ or $f = c$) need not be included.

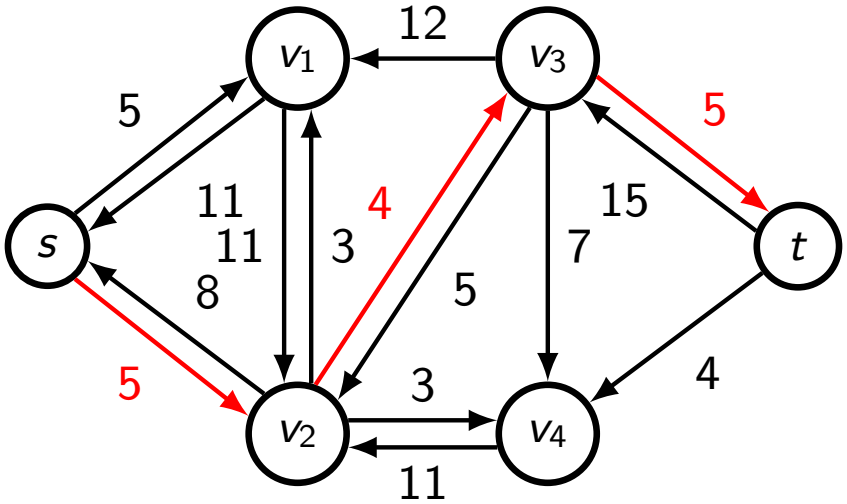
Residual Flow Network

- Suppose the original flow network has an edge from v to w with capacity c_1 and flow f_1 units, <u>and</u> an edge from w to v with capacity c_2 and flow f_2 units.
- What are the corresponding edges in the residual graph?
- How much flow can be sent from v to w (and vice versa)?
- The forward edge allows $c_1 - f_1$ additional units of flow, and we can also send up to f_2 units to cancel the flow through the reverse edge.
- Thus we create edges from v to w with capacity $c_1 - f_1 + f_2$ and similarly from w to v with capacity $c_2 - f_2 + f_1$.

Definition

An augmenting path is a path from s to t in the residual flow network.

The residual flow network below corresponds to the earlier example of a flow of value 19 units. An augmenting path is pictured in red.

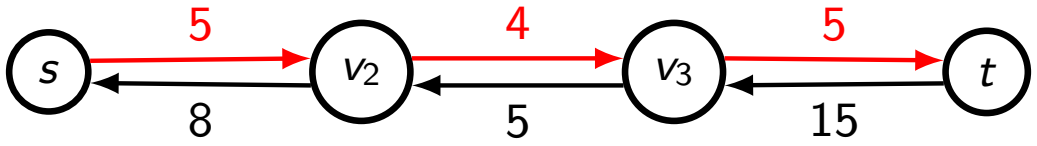


Question. What's the maximum extra flow we can send along this red path? 4 units.
Adding this to our 19-unit flow gives a flow of 23 units.

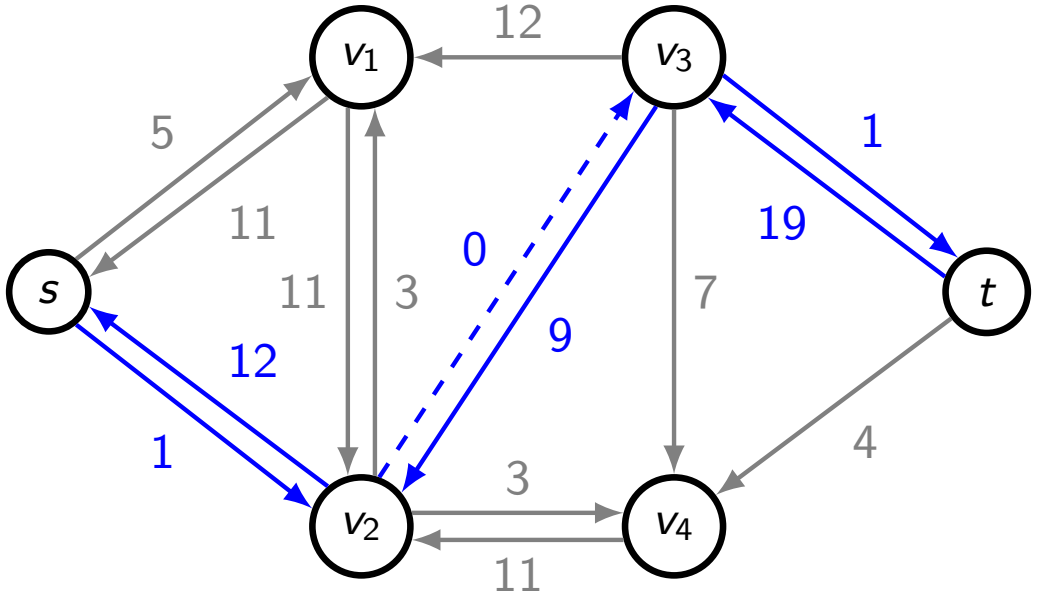
Augmenting Paths

- The capacity of an augmenting path is the capacity of its "bottleneck" edge, i.e., the edge of smallest capacity.
- We can now send that amount of flow along the augmenting path, recalculating the flow and the residual capacities for each edge used.
- Suppose we have an augmenting path of capacity f, including an edge from v to w. We should:
  - cancel up to f units of flow being sent from w to v,
  - add the remainder of these f units to the flow being sent from v to w,
  - increase the residual capacity from w to v by f, and
  - reduce the residual capacity from v to w by f.

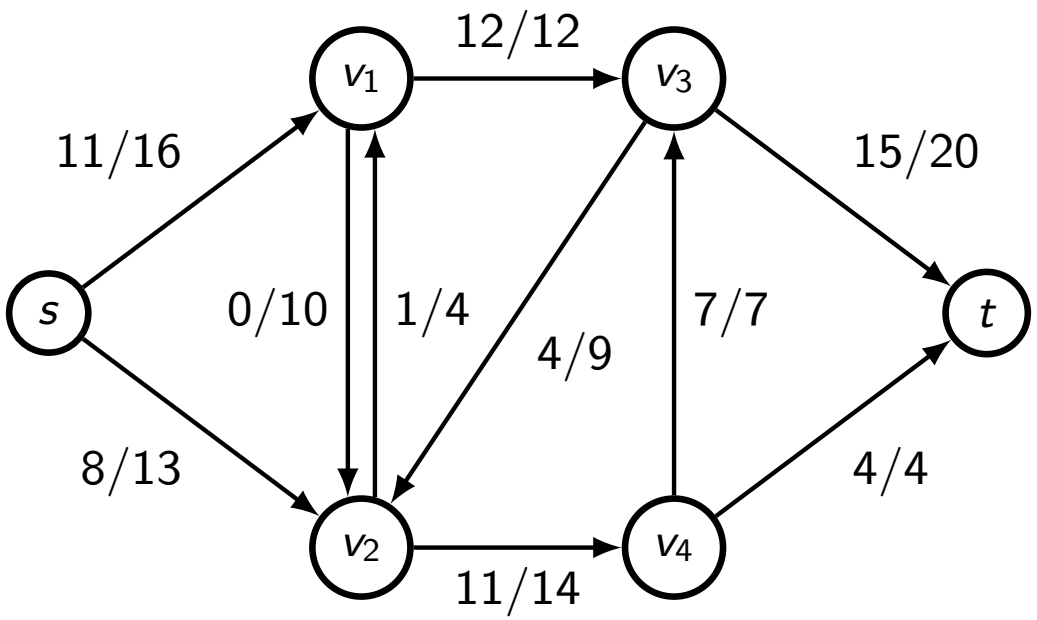
Recall that the augmenting path was as follows.



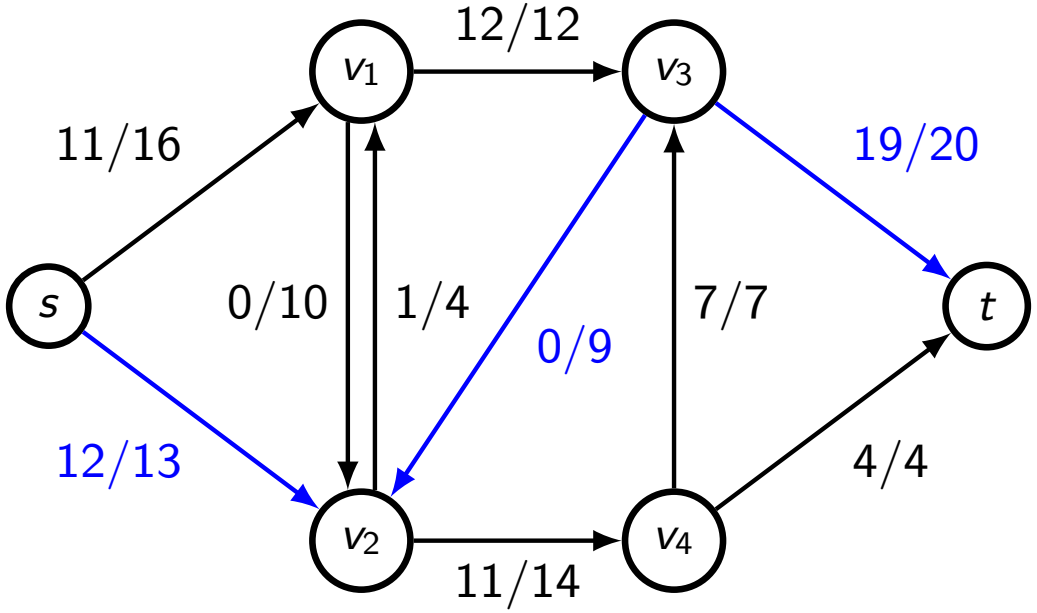
After sending four units of flow along this path, the new residual flow network is pictured below.



The flow used to be as follows.



Pictured below is the new flow, after sending four units of flow along the path $s \to v_2 \to v_3 \to t$.



Note that the four units of flow previously sent from v_3 to v_2 have been cancelled out.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

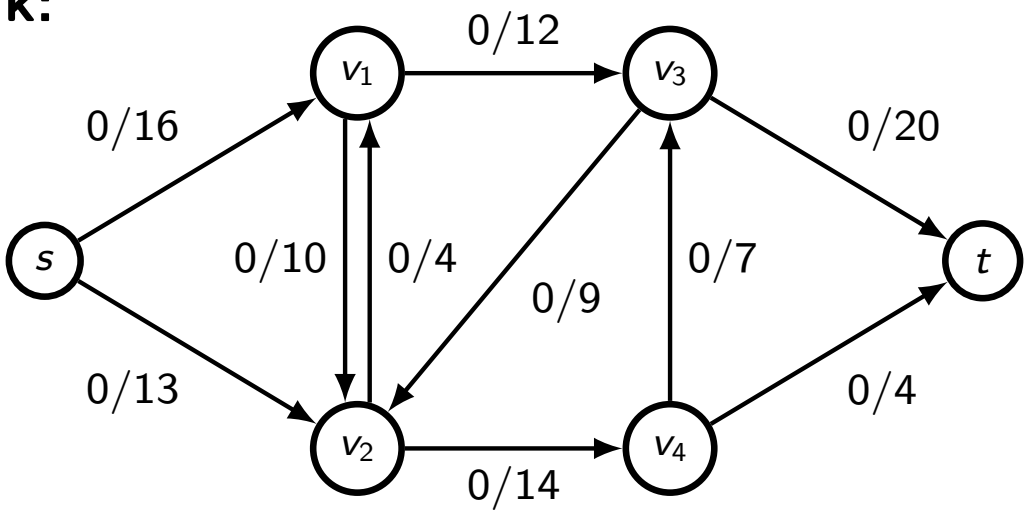
7. Puzzle

Algorithmic approach:

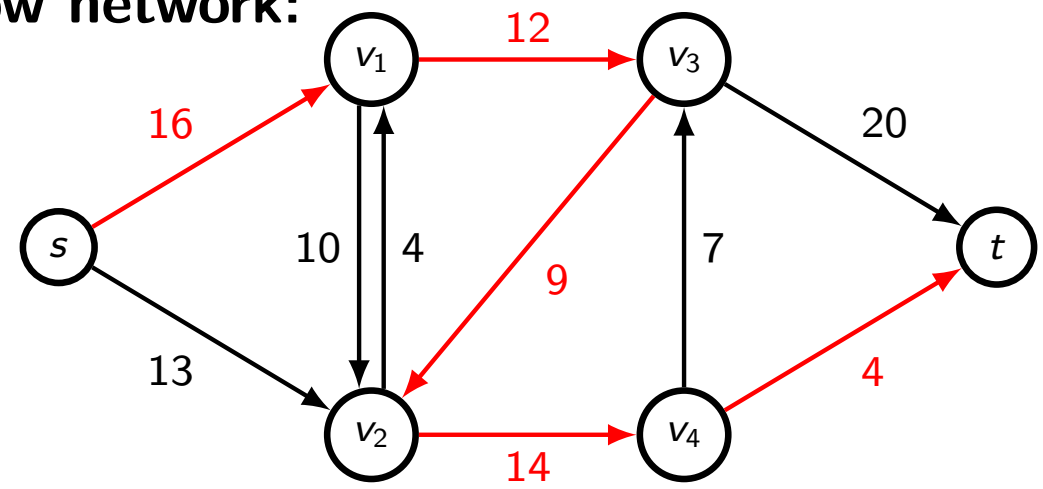
- Keep adding flow through new augmenting paths for as long as it is possible.

- When there are no more augmenting paths, you have achieved the largest possible flow in the network.

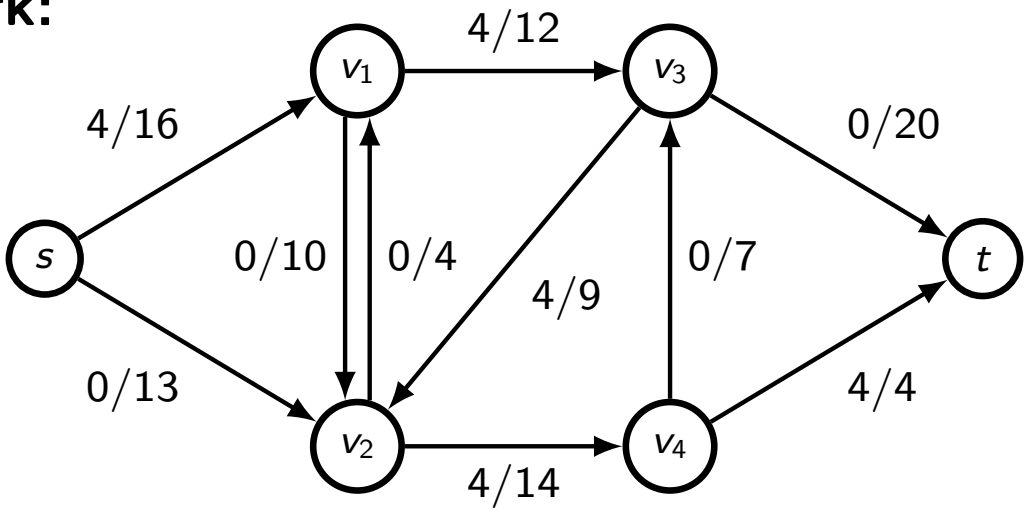
Flow network:



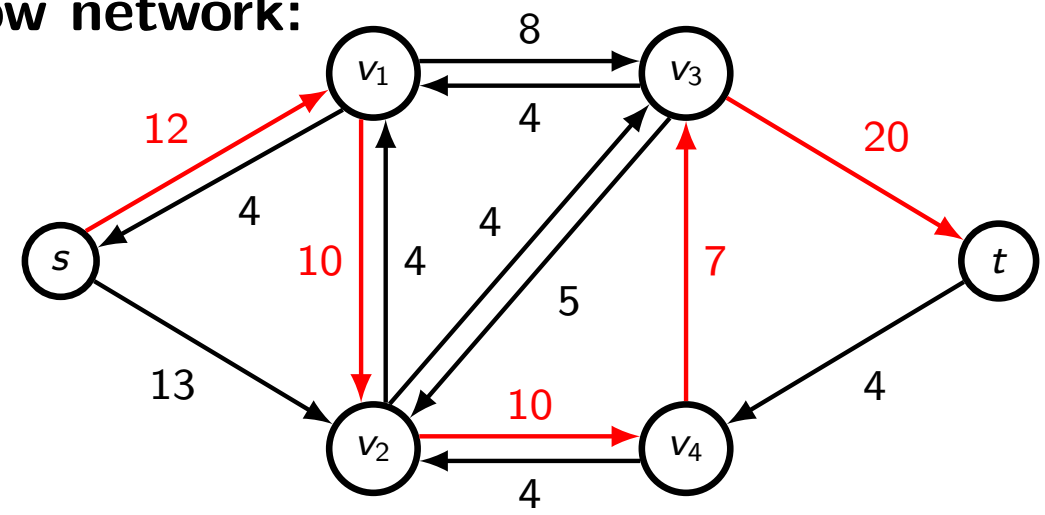
Residual flow network:



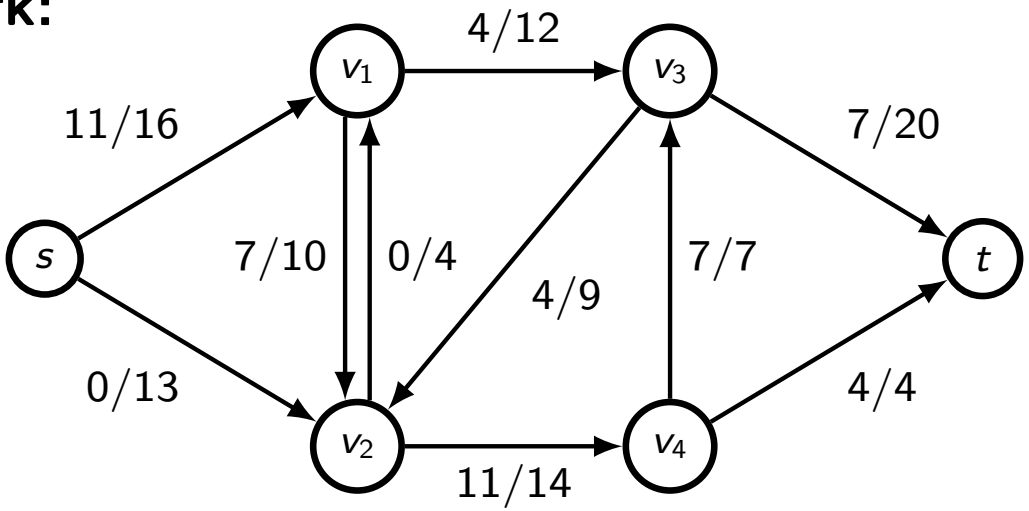
Flow network:



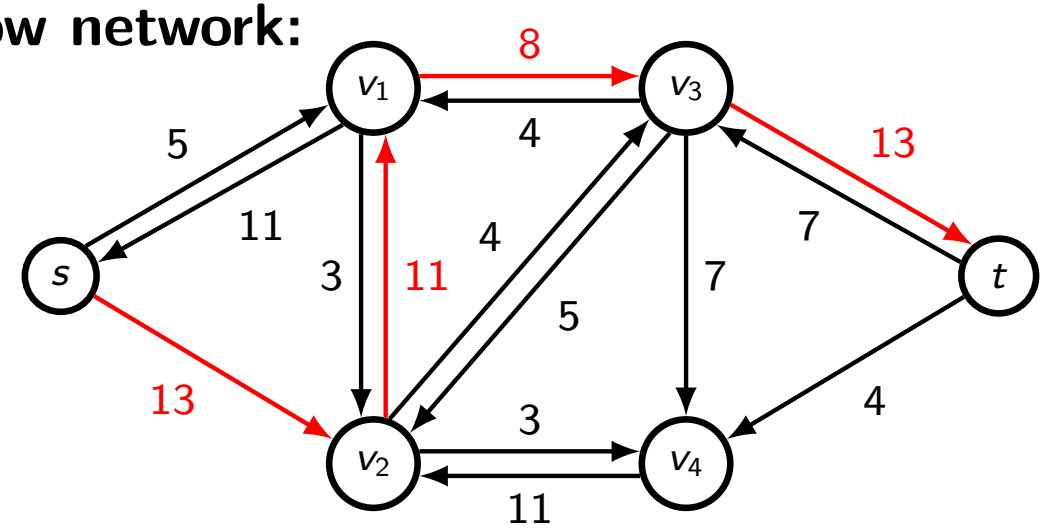
Residual flow network:



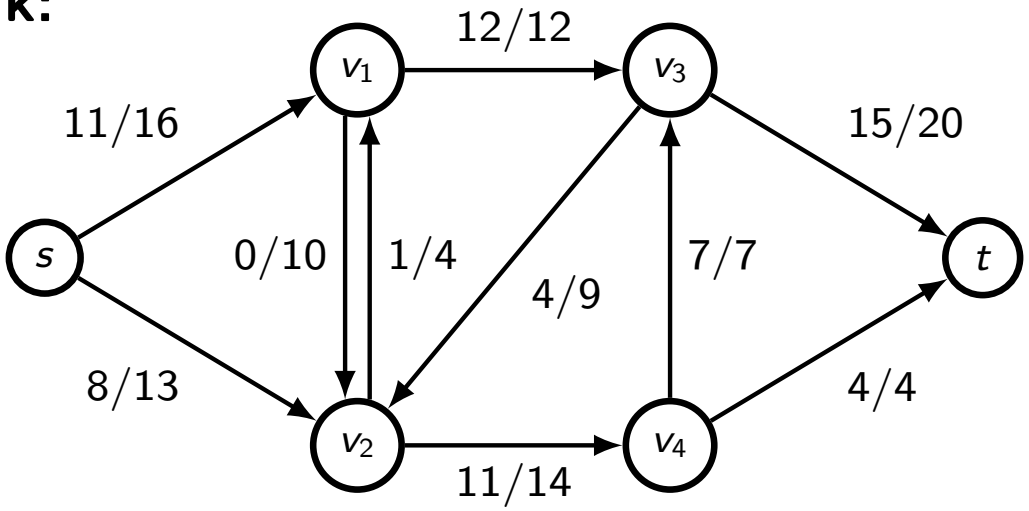
Flow network:



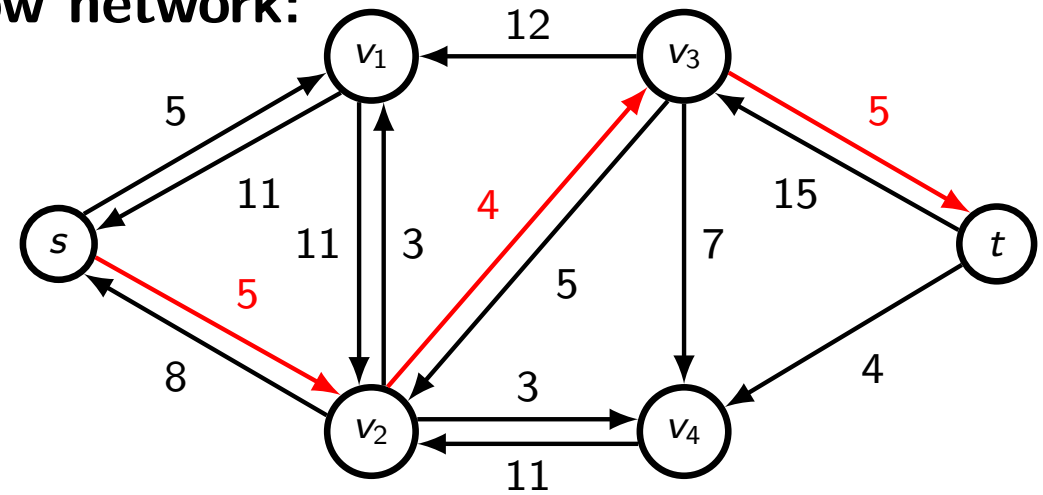
Residual flow network:



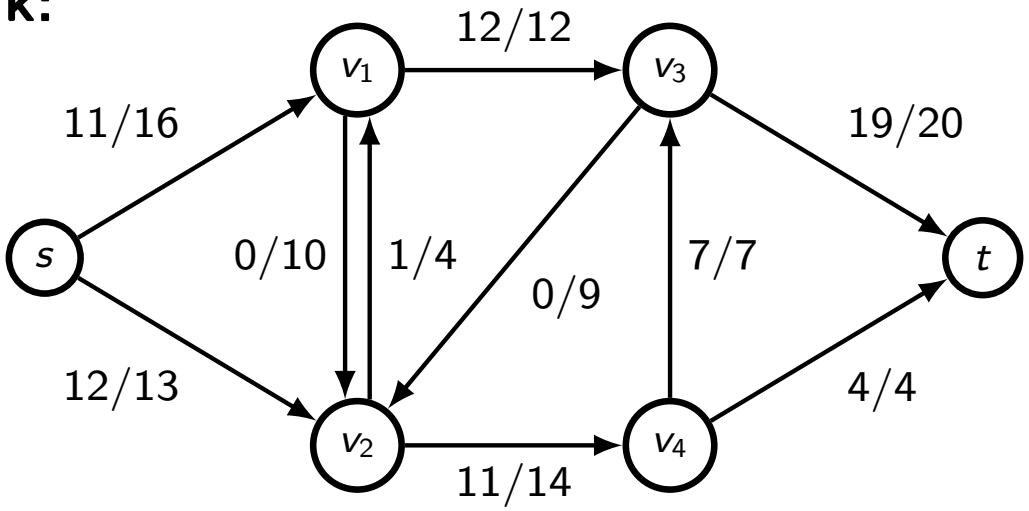
Flow network:



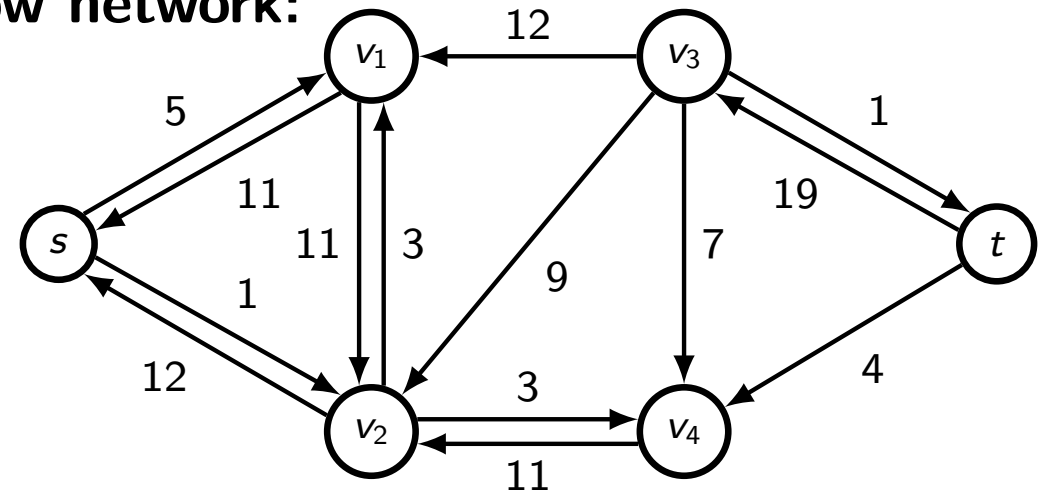
Residual flow network:



Flow network:



Residual flow network:



Question. Why does this procedure terminate?

In other words, why can't we get stuck in a loop, which keeps adding augmenting paths forever?

- If all the capacities are integers, then each augmenting path increases the flow through the network by at least 1 unit.
- However, the total flow is finite. In particular, it cannot be larger than the sum of all capacities of all edges leaving the source.
- We conclude that the process must terminate eventually.

Question. Why does this procedure give a maximum flow?

- Even if the procedure does terminate, why does it produce a flow of the largest possible value?
- Maybe we have created bottlenecks by choosing bad augmenting paths; maybe better choices of augmenting paths could produce a larger total flow through the network?
- This is not at all obvious, and to show that this is not the case we need a mathematical proof!

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Cuts in a Flow Network

The proof is based on the notion of a minimum cut in a flow network.

Definition

A <u>cut</u> in a flow network is any partition of the vertices of the underlying graph into two subsets S and T such that:

1. $S \cup T = V$
2. $S \cap T = \emptyset$
3. $s \in S$ and $t \in T$.

Definition

The capacity $c(S, T)$ of a cut (S, T) is the sum of capacities of all edges leaving S and entering T, i.e.

$$c(S, T) = \sum_{(u,v) \in E} \{c(u, v) : u \in S, v \in T\}.$$

Note that the capacities of edges going in the opposite direction, i.e., from T to S, do not count.

Definition

Given a flow f, the <u>flow</u> $f(S, T)$ through a cut (S, T) is the total flow through edges from S to T minus the total flow through edges from T to S, i.e.,

$$f(S, T) = \sum_{(u,v) \in E} \{f(u, v) : u \in S, v \in T\} - \sum_{(u,v) \in E} \{f(u, v) : u \in T, v \in S\}.$$

Exercise

Prove that for any flow f, the flow through any cut (S, T) is equal to the value of the flow, i.e.

$$f(S, T) = |f|.$$

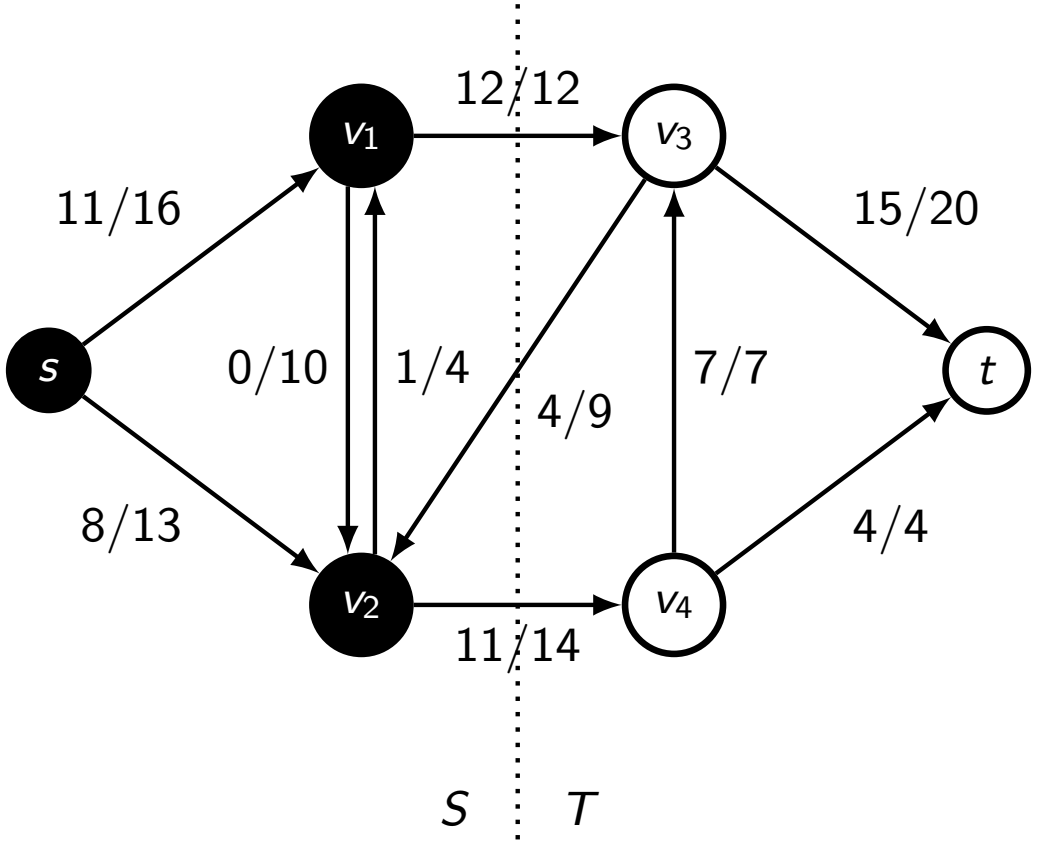
Hint

Recall the definition of the value of a flow, and use the property of flow conservation.

Cuts in a Flow Network

Question. Given a cut (S, T), how does the flow of the cut compare to its capacity?

- An edge from S to T counts its full capacity towards $c(S, T)$, but only the flow through it towards $f(S, T)$.
- An edge from T to S counts zero towards $c(S, T)$, but counts the flow through it <u>in the negative</u> towards $f(S, T)$.
- Therefore, $f(S, T) \leq c(S, T)$.
- It follows that $|f| \leq c(S, T)$, so the value of any flow is at most the capacity of any cut.



Question. What is the capacity of this cut? $12 + 14 = 26$.
Question. What is the flow through this cut? (12+11) - (4) $= 19$.

Cuts in a Flow Network: Example

Writing this more formally:

- In the previous slide, the **capacity** of the cut $c(S, T)$ is given by
$$c(S, T) = c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26.$$
- As we have mentioned, we add only the capacities of edges from S to T and not of edges in the opposite direction.
- The **flow** through the cut is given by
$$f(S, T) = f(v_1, v_3) + f(v_2, v_4) - f(v_2, v_3) = 12 + 11 - 4 = 19.$$
- Note that the flow in the opposite direction (from T to S) is subtracted.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Theorem

The maximum amount of flow in a flow network is equal to the capacity of the cut of minimum capacity.

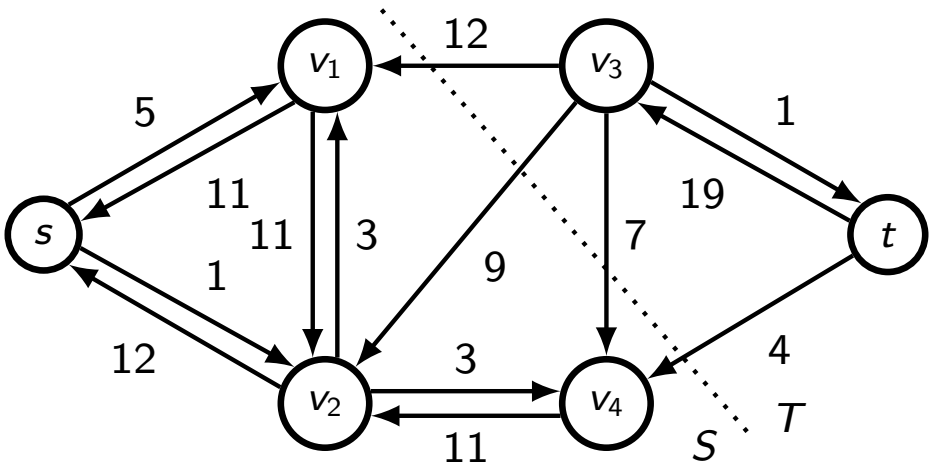
Proof idea:

- Let f be a flow. Recall that the value $|f|$ is at most the capacity of any cut: $c(S, T)$.
- Thus, if we find a flow f which equals the capacity of some cut (S, T), then such flow must be maximum and the capacity of such a cut must be minimum.
- We now show that when the Ford-Fulkerson algorithm terminates, it produces a flow equal to the capacity of an appropriately defined cut.

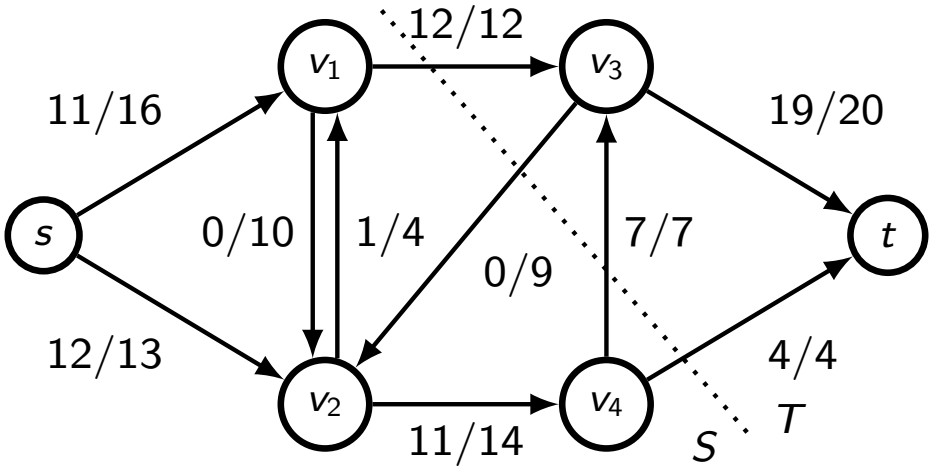
Proof of correctness of FFA / Proof of max-flow min-cut:

- Assume that the Ford-Fulkerson algorithm has terminated, so there no more augmenting paths from the source s to the sink t in the last residual flow network.
- Define S to be the source s and all vertices u such that there is a (directed) path in the residual flow network from the source s to that vertex u.
- Define T to be the set of all vertices for which there is no such path.
- Since there are no more augmenting paths from s to t, clearly the sink t belongs to T.

Final residual flow network (no augmenting paths remain):



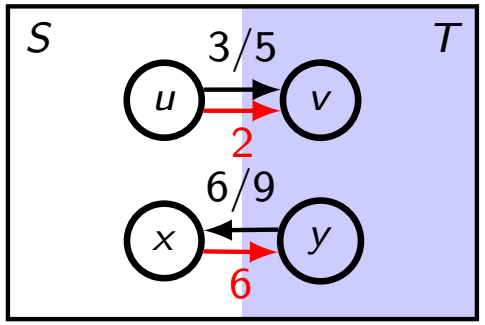
Corresponding flow network (shows solution found by FFA):



Claim

All the edges from S to T are fully occupied with flow (i.e. we cannot add more flow), and all the edges from T to S are empty.

Question. Suppose S and T were chosen according to the rule on slide 35. Why can't our (residual) flow network contain the edges below?



Answer. If (u, v) did not use its full capacity (5, in black), then it would have a positive flow in the residual flow network (red). So we would have placed v into S instead of T.
The argument for (y, x) is similar.

We now use that intuition to prove the claim.

Proof of claim

- Suppose an edge (u, v) from S to T has some additional capacity left. Then in the residual flow network, the path from s to u could be extended to a path from s to v. This contradicts our assumption that $v \in T$.
- Suppose an edge (y, x) from T to S has some flow in it. Then in the residual flow network, the path from s to x could be extended to a path from s to y. This contradicts our assumption that $y \in T$.

Ford-Fulkerson Algorithm: Proof of correctness (4 of 4)

We now use the claim to prove the correctness of the Ford-Fulkerson Algorithm

Proof of correctness of FFA

- By the claim, we know that all edges from S to T are occupied with flows equal to their maximum capacity, and we also know that there is no flow from T to S.
- So the flow through the cut (S, T) is precisely equal to the capacity of this cut, giving

$$f(S, T) = c(S, T).$$

- Thus, such a flow is a <u>maximum flow</u>, and the corresponding cut is a <u>minimum cut</u>, regardless of the particular way in which the augmenting paths were chosen.
- So the Ford-Fulkerson algorithm has output a maximum flow.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

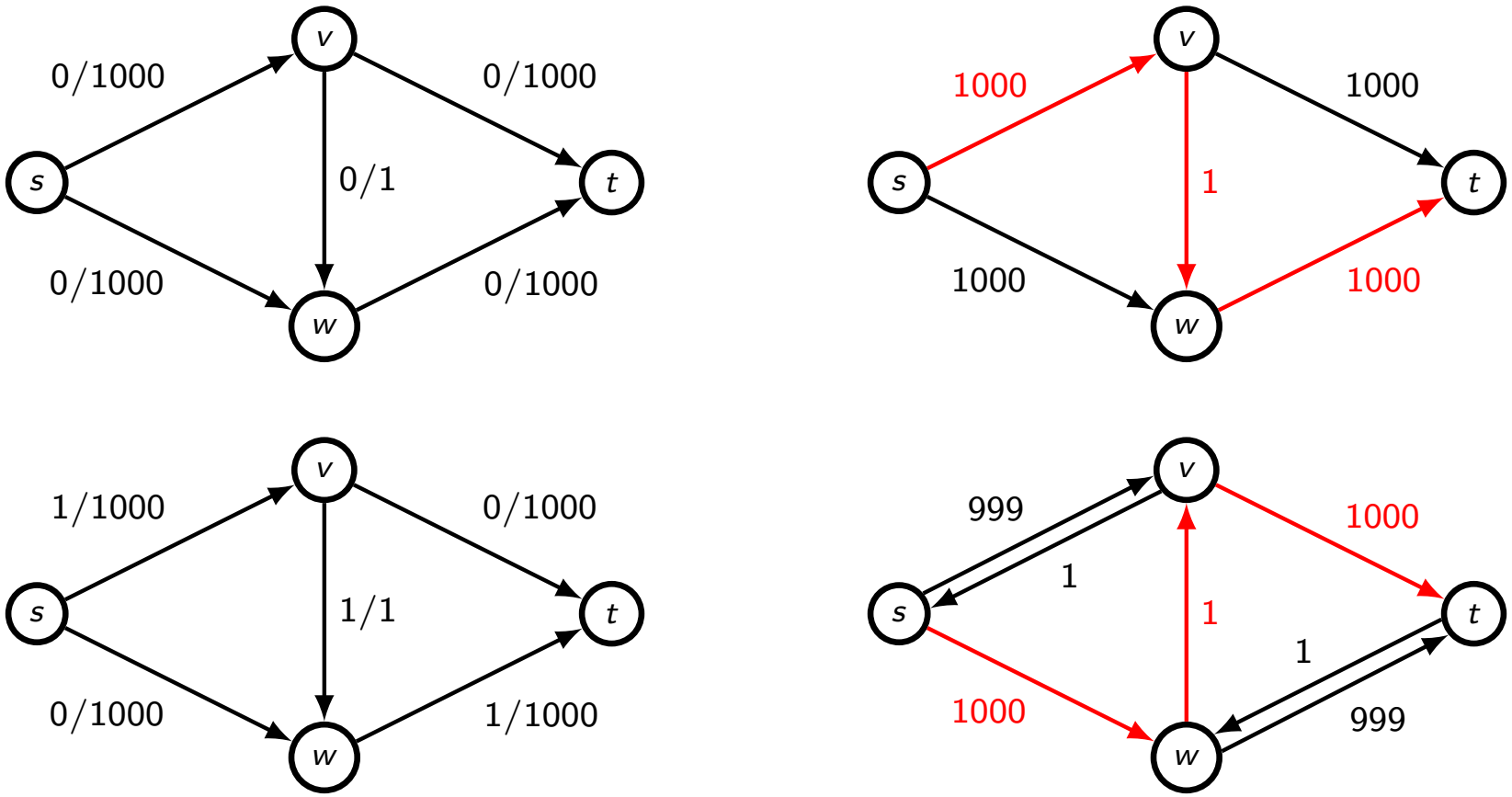
5. Minimum Cost Maximum Flow

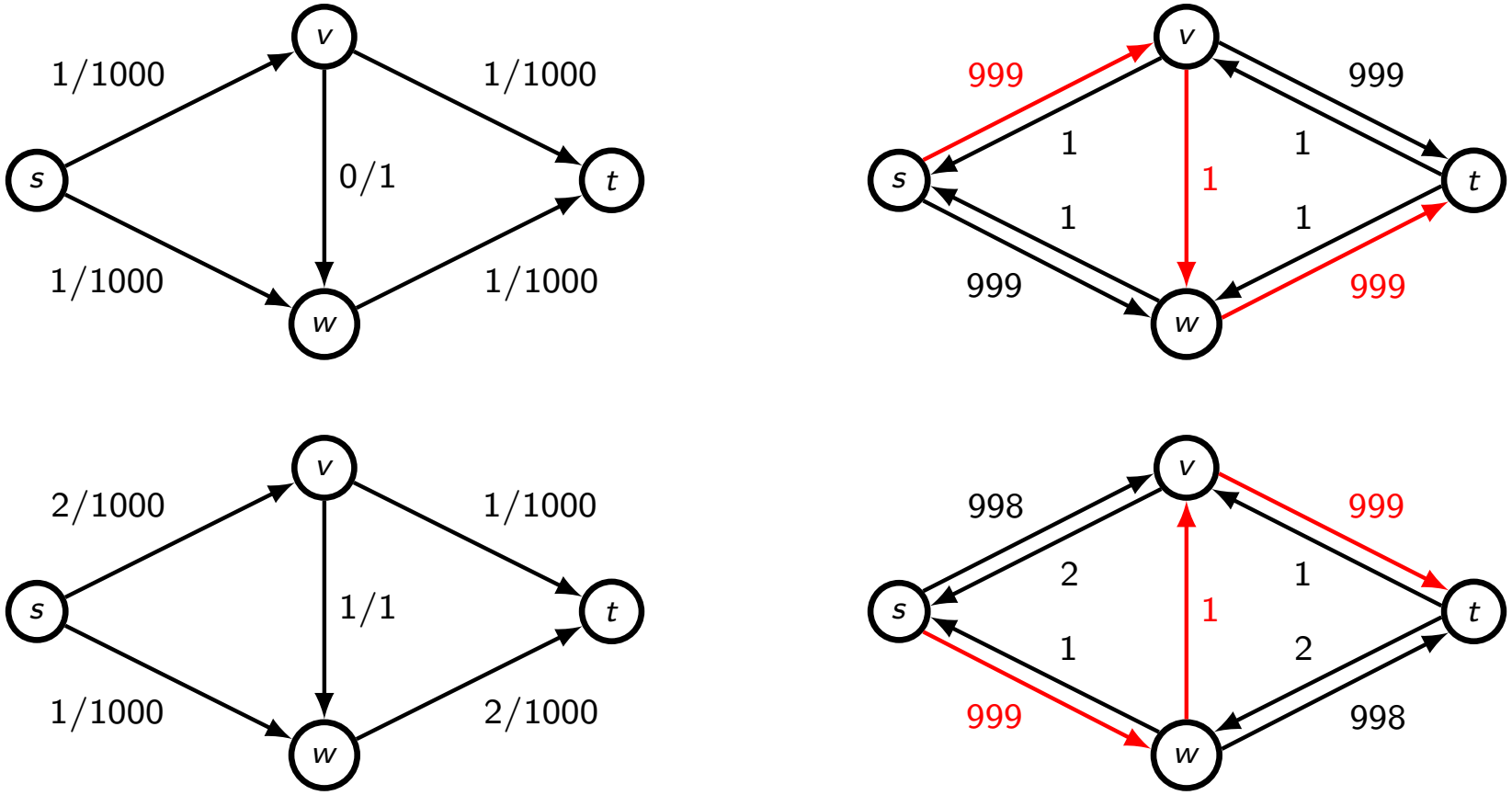
6. Matroid Intersection

7. Puzzle

Question. How efficient is the Ford-Fulkerson algorithm?

Example.





If we choose augmenting paths arbitrarily, then poor choices can lead to big running times.

Ford-Fulkerson Algorithm: Time Complexity

Question. How large can the running time get?

- The number of augmenting paths can be up to the value of the max flow, denoted $|f|$.
- Each augmenting path is found in $O(|E|)$ time, e.g., by a DFS from the source s.
- Therefore, the time complexity of the Ford-Fulkerson algorithm is $O(|E| \cdot |f|)$.

Question. How does this compare to the size of the input?

- Recall that the input specifies $|V|$ vertices and $|E|$ edges, as well as a capacity for each edge.
- If the capacities are all $\leq C$, then the length of the input (i.e. the number of bits required to encode it) is $O(|V| + |E| \log C)$.
- However, the value of the maximum flow $|f|$ can be as large as $|E| \cdot C$.
  - Even if we forbid parallel edges, it can be up to $|E| \cdot C/2$: imagine a graph where for each vertex $v \in V \setminus \{s, t\}$, there are directed edges (s, v) and (v, t) with capacity C.
- Therefore, the time complexity $O(|E| \cdot |f|)$ can be exponential in the size of the input. This is very slow.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Edmonds-Karp Algorithm

Question. How can we adapt Ford-Fulkerson to ensure it always chooses "good" augmenting paths?

- The Edmonds-Karp algorithm improves the Ford-Fulkerson algorithm in a simple way: always choose the augmenting path which consists of the fewest edges.

- At each step, we find the next augmenting path using breadth-first search in $O(|E|)$ time.

- Note that this choice is somewhat counter-intuitive: augmenting paths are chosen based only on length, so we may end up flowing edges with small capacities before edges with large capacities.

Question

What is the time complexity of the Edmonds-Karp algorithm?

Answer

It can be proven (see CLRS pp.727–730) that the number of augmenting paths is $O(|V| \cdot |E|)$, and since each takes $O(|E|)$ to find, the time complexity is $O(|V| \cdot |E|^2)$.

Note also that Edmonds-Karp is a specialisation of Ford-Fulkerson, so the original $O(|E| \cdot |f|)$ bound also applies.

Faster max flow algorithms

- Faster max flow algorithms exist, e.g., but we will <u>not</u> allow them in assessments in this course
  - $O(|E|^{1+o(1)} \log C)$ time, where C is the largest capacity, by van den Brand et al. (FOCS 2023)
  - $O(|V| \cdot |E|)$ by Orlin (STOC 2013)

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Real-world networks

The flow networks we've considered so far satisfied the following properties:

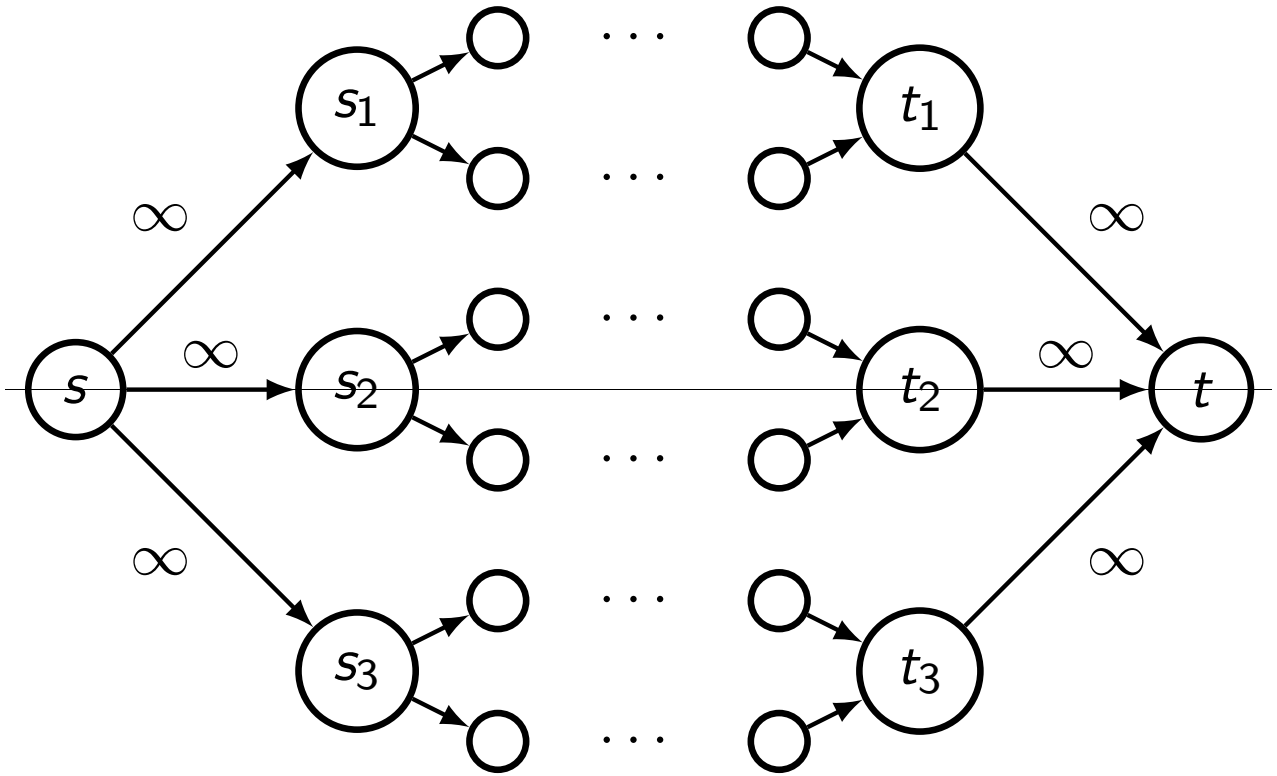
- They had exactly one source vertex and exactly one sink vertex.
- The edges had capacities.

Real-world problems may be more complicated. For example, they may include:

- Multiple sources and/or multiple sinks.
- Capacities at vertices.

These seemingly more complicated problems can also be modelled by the simpler networks that we've already seen.

Flow networks with multiple sources and sinks are reducible to networks with a single source and single sink by adding a "super-source" and "super-sink" and connecting them to all sources and from all sinks respectively.



Networks with multiple sources and sinks

- If there is a constraint on how much flow can originate at each of the sources or be collected at each of the sinks, we can encode it in the capacities of the edges we construct.
- Otherwise, we use edges of infinite capacity.
  - An infinite capacity edge will never be saturated; in the residual graph, the forwards edge has infinite capacity no matter how much flow has already been sent forwards.
  - So long as there isn't an infinite capacity path from s to t, the usual time complexity bounds still apply for Edmonds-Karp.
  - Practically not much different than choosing a finite edge capacity much much larger than the rest of the capacities.

Networks with vertex capacities

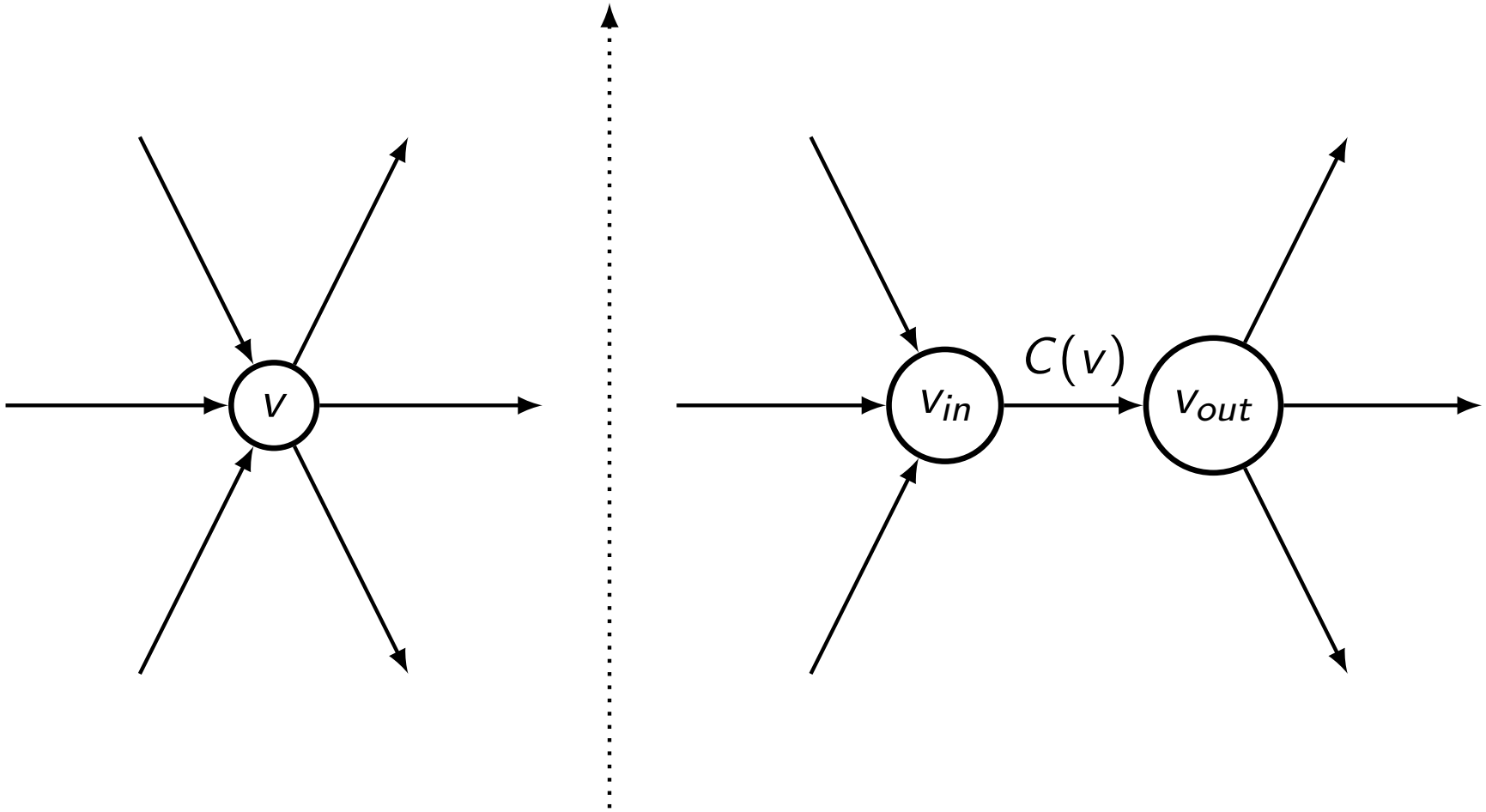
- Sometimes not only the edges but also the vertices v_i of the flow graph might have capacities $C(v_i)$, which limit the total throughput of the flow coming to the vertex (and, consequently, also leaving the vertex):

$$\sum_{e(u,v)\in E} f(u,v) = \sum_{e(v,w)\in E} f(v,w) \leq C(v).$$

- We can handle this by reducing it to a situation with only edge capacities!

Networks with vertex capacities

- Suppose vertex v has capacity $C(v)$.
- Split v into two vertices v_{in} and v_{out}.
- Attach all of v's incoming edges to v_{in} and all its outgoing edges from v_{out}.
- Connect v_{in} and v_{out} with an edge $e^* = (v_{in}, v_{out})$ of capacity $C(v)$.


v
v_{in}
$C(v)$
v_{out}

Example problem: Movie Rental

Problem

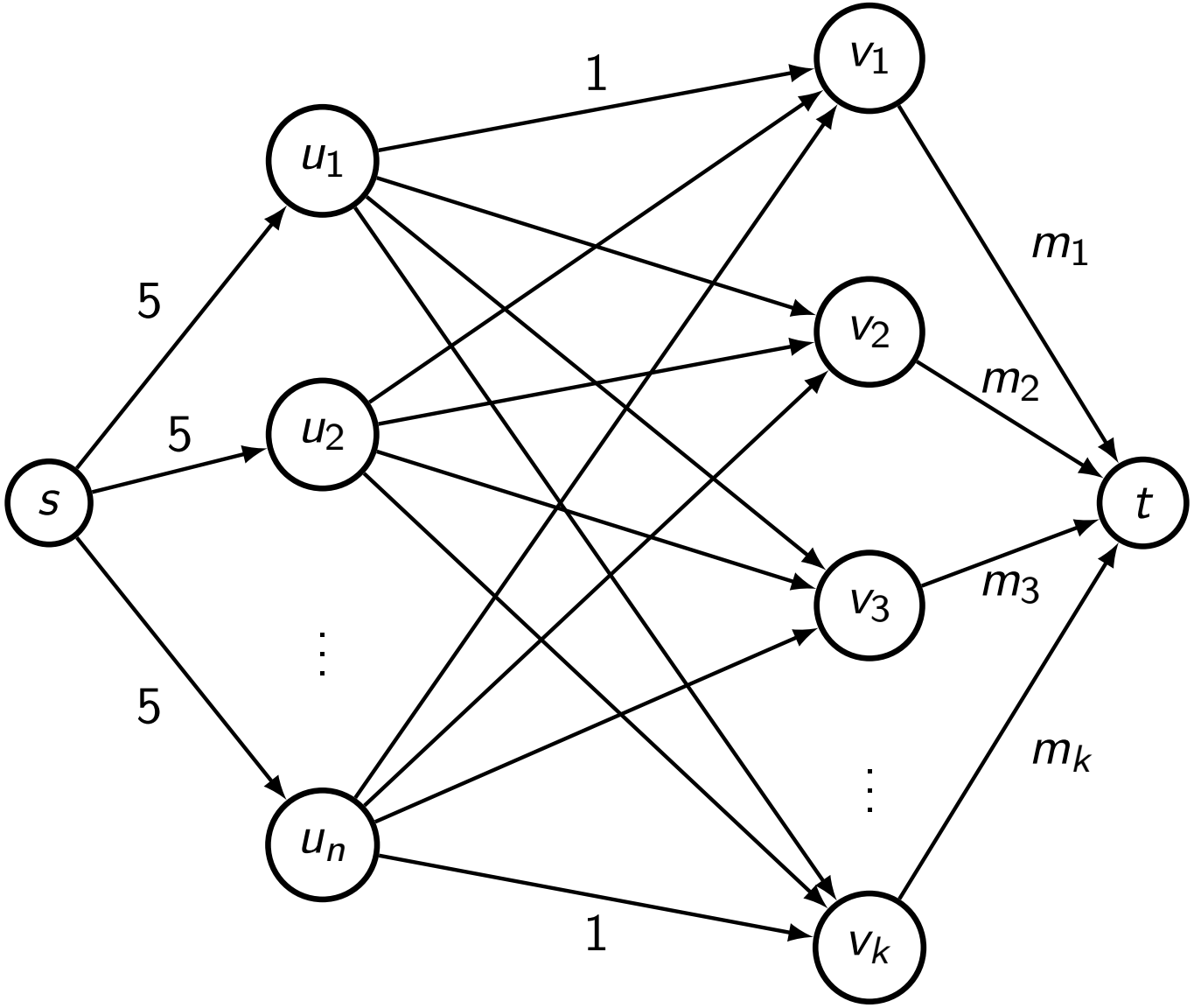
Instance: Suppose you have a movie rental agency.
You have k movies in stock, with m_i copies of movie i.
There are n customers, who have each specified which subset of the k movies they are willing to see. However, no customer can rent out more than 5 movies at a time.
Task: Design an algorithm which runs in $O(n^2 k)$ time and dispatches the largest possible number of movies.

Example problem: Movie Rental

Construct a flow network with:

- source s and sink t,
- a vertex u_i for each customer i.
- a vertex v_j for each movie j,
- for each customer i, an edge from s to u_i with capacity 5,
- for each customer i, and for each movie j that they are willing to see, an edge from u_i to v_j with capacity 1.
- for each movie j, an edge from v_j to t with capacity m_j.

Example problem: Movie Rental



Question. What does a flow in this network represent?

- Each customer-movie edge has capacity 1, so we will interpret a flow of 1 from u_i to v_j as assigning movie j to customer i.
- Each customer only receives movies that they want to see.
- By flow conservation, the amount of flow sent along the edge from s to u_i is equal to the total flow sent from u_i to all movie vertices v_j, so it represents the number of movies received by customer i. Again, the capacity constraint ensures that this does not exceed 5 as required.
- Similarly, the movie stock levels m_j are also respected.
- Therefore, any flow in this graph corresponds to a valid allocation of movies to customers.

Question. What is the running time to find the maximum flow?

- To maximise the movies dispatched, we find the maximum flow using the Edmonds-Karp algorithm.
- There are $n + k + 2$ vertices and up to $nk + n + k$ edges, so the time complexity is

$$O(|V| \cdot |E|^2) = O((n + k + 2)(nk + n + k)^2).$$

- Since the value of any flow is constrained by the total capacity from s, which in this case is $5n$, we can achieve a tighter bound of

$$O(|E| \cdot |f|) = O((nk + n + k)n) = O(n^2 k).$$

Problem

Instance: The storage space of a ship is in the form of a rectangular grid of cells with n rows and m columns. Some of the cells are taken by support pillars and cannot be used for storage, so they have 0 capacity.

You are given the capacity of every cell; the cell in row i and column j has capacity $C(i, j)$. To ensure the stability of the ship, the total weight in each row i must not exceed $C_r(i)$ and the total weight in each column j must not exceed $C_c(j)$.

Task: Design an algorithm which runs in $O((n + m)(nm)^2)$ time and allocates the maximum possible weight of cargo without exceeding any of the cell, row or column capacities.

Example problem: Cargo Allocation

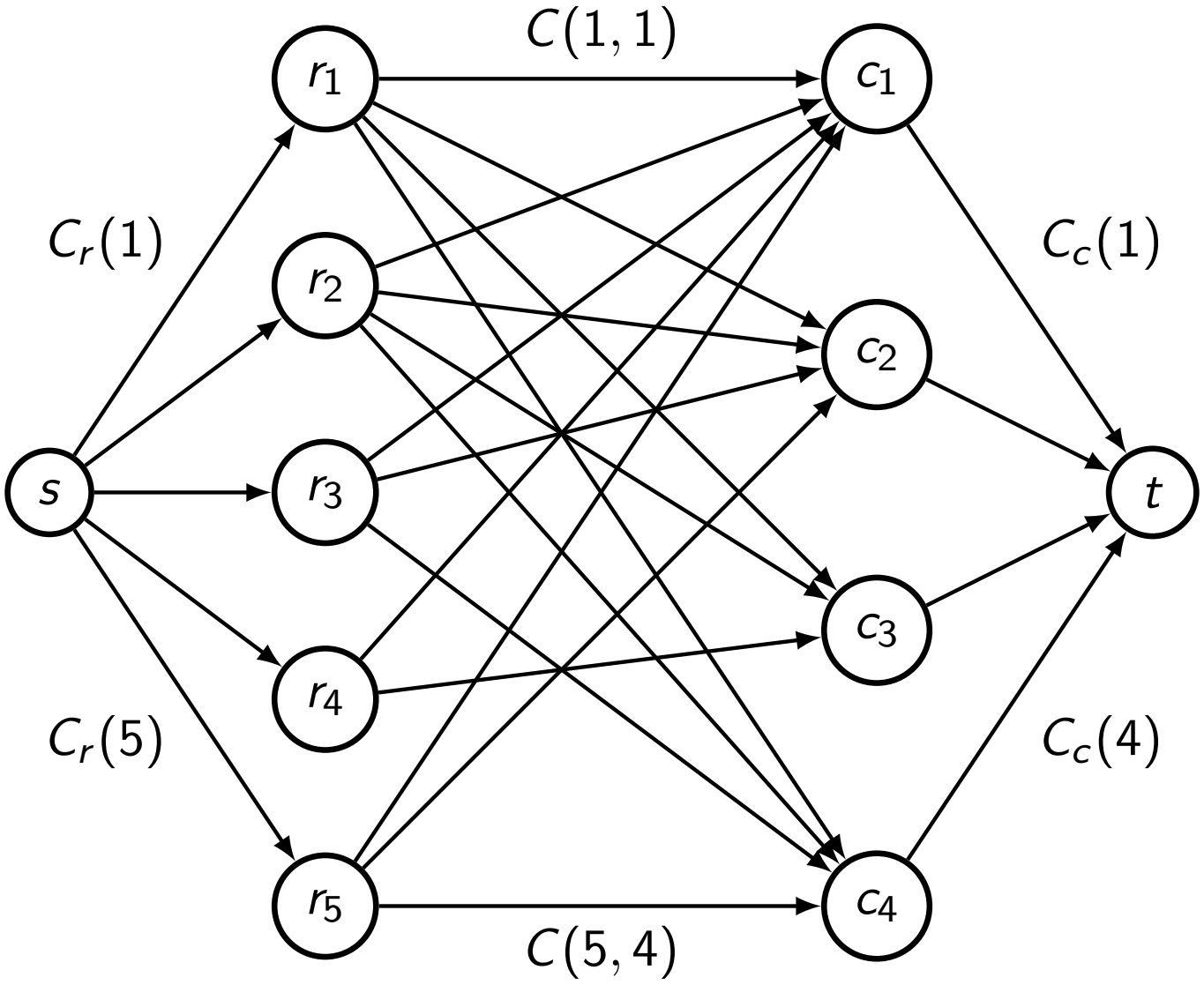
	col 1	col 2	col 3	col 4	row cap
row 1	$C(1,1)$	$C(1,2)$	$C(1,3)$	$C(1,4)$	$C_r(1)$
row 2	$C(2,1)$	$C(2,2)$	$C(2,3)$	$C(2,4)$	$C_r(2)$
row 3	$C(3,1)$	$C(3,2)$	0	$C(3,4)$	$C_r(3)$
row 4	$C(4,1)$	0	$C(4,3)$	0	$C_r(4)$
row 5	$C(5,1)$	$C(5,2)$	0	$C(5,4)$	$C_r(5)$
col cap	$C_c(1)$	$C_c(2)$	$C_c(3)$	$C_c(4)$	

Example problem: Cargo Allocation

Construct a flow network with:

- source s and sink t,
- a vertex r_i for each row i and a vertex c_j for each column j,
- for each i, an edge from s to r_i with capacity $C_r(i)$,
- for each cell (i, j) which is not a pillar, an edge from r_i to c_j with capacity $C(i, j)$.
- for each j, an edge from c_j to t with capacity $C_c(j)$.

Example problem: Cargo Allocation



Question. What does a flow in this network represent?

- We will interpret the amount of flow sent along the edge from r_i to c_j as the weight of cargo stored in cell (i, j).
- By the capacity constraint, this weight cannot exceed the edge capacity $C(i, j)$, so the weight limit of each cell is satisfied.
- By flow conservation, the amount of flow sent along the edge from s to r_i is equal to the total flow sent from r_i to all column vertices c_j, so it represents the total weight stored in row i. Again, the capacity constraint ensures that this does not exceed $C_r(i)$ as required.
- Similarly, the column capacities $C_c(j)$ are also respected.
- Therefore, any flow in this graph corresponds to a valid allocation of cargo to cells.

Question. What is the running time to find the maximum flow?

- To maximise the cargo allocated, we find the maximum flow using the Edmonds-Karp algorithm.

- There are $n + m + 2$ vertices and up to $nm + n + m$ edges, so the time complexity is

$$\begin{aligned} O(|V| \cdot |E|^2) &= O((n + m + 2)(nm + n + m)^2) \\ &= O((n + m)(nm)^2), \end{aligned}$$

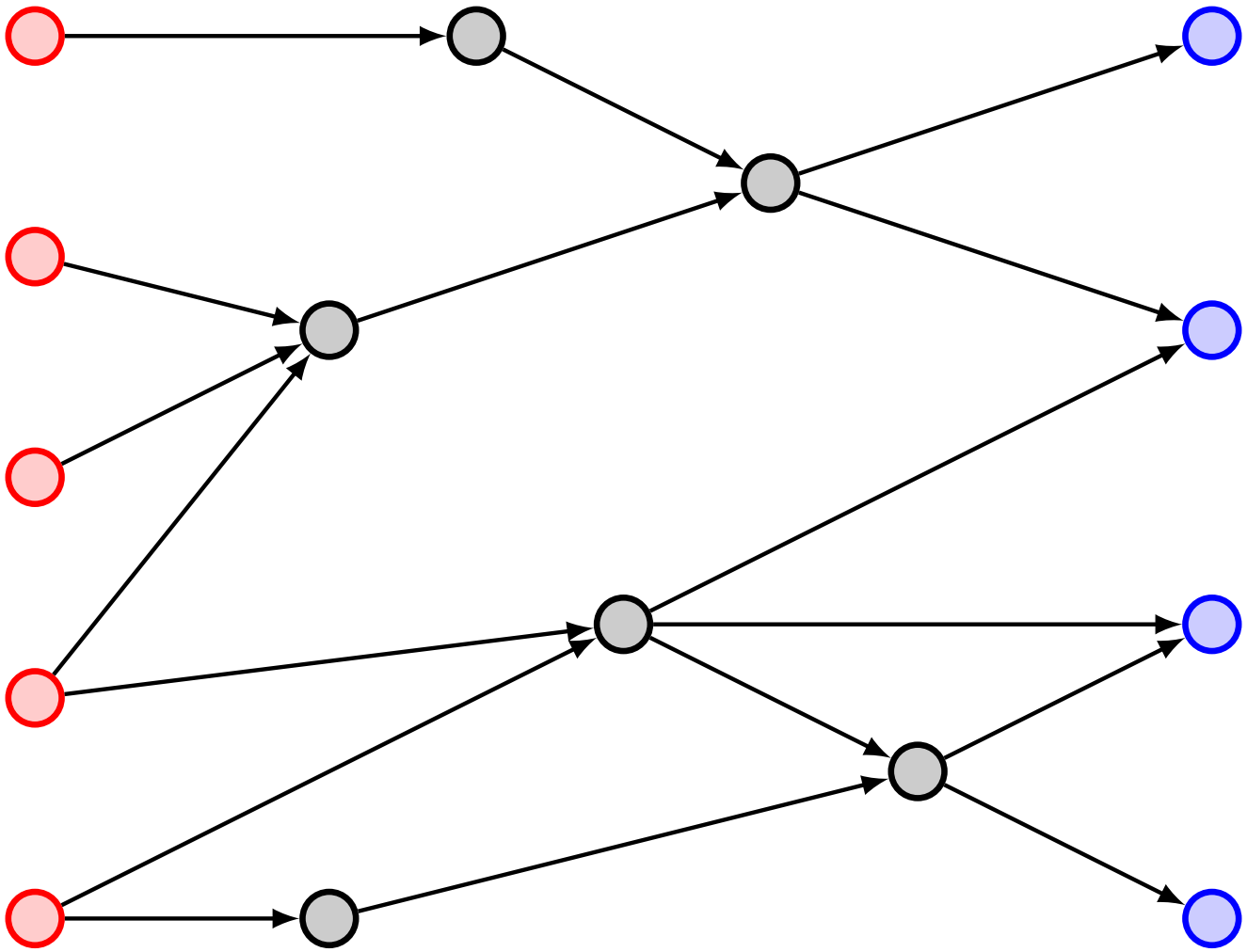
as required.

Example problem: Vertex-Disjoint Paths

Problem

Instance: You are given a directed graph G with n vertices and m edges. Of these vertices, r are painted red, b are painted blue, and the remaining $n - r - b$ are black. Red vertices have only outgoing edges and blue vertices have only incoming edges.
Task: Design an algorithm which runs in $O(n(n + m))$ time and determines the largest possible number of vertex-disjoint (i.e. non-intersecting) paths in this graph, each of which starts at a red vertex and finishes at a blue vertex.

Example problem: Vertex-Disjoint Paths



Example problem: Vertex-Disjoint Paths

Observations.

- The red vertices function as sources, and the blue vertices as sinks.
- To use our usual max flow algorithms, we need to introduce a super-source and super-sink.
- To ensure that no black vertex is used twice, we should impose a vertex capacity of 1 for each of them.

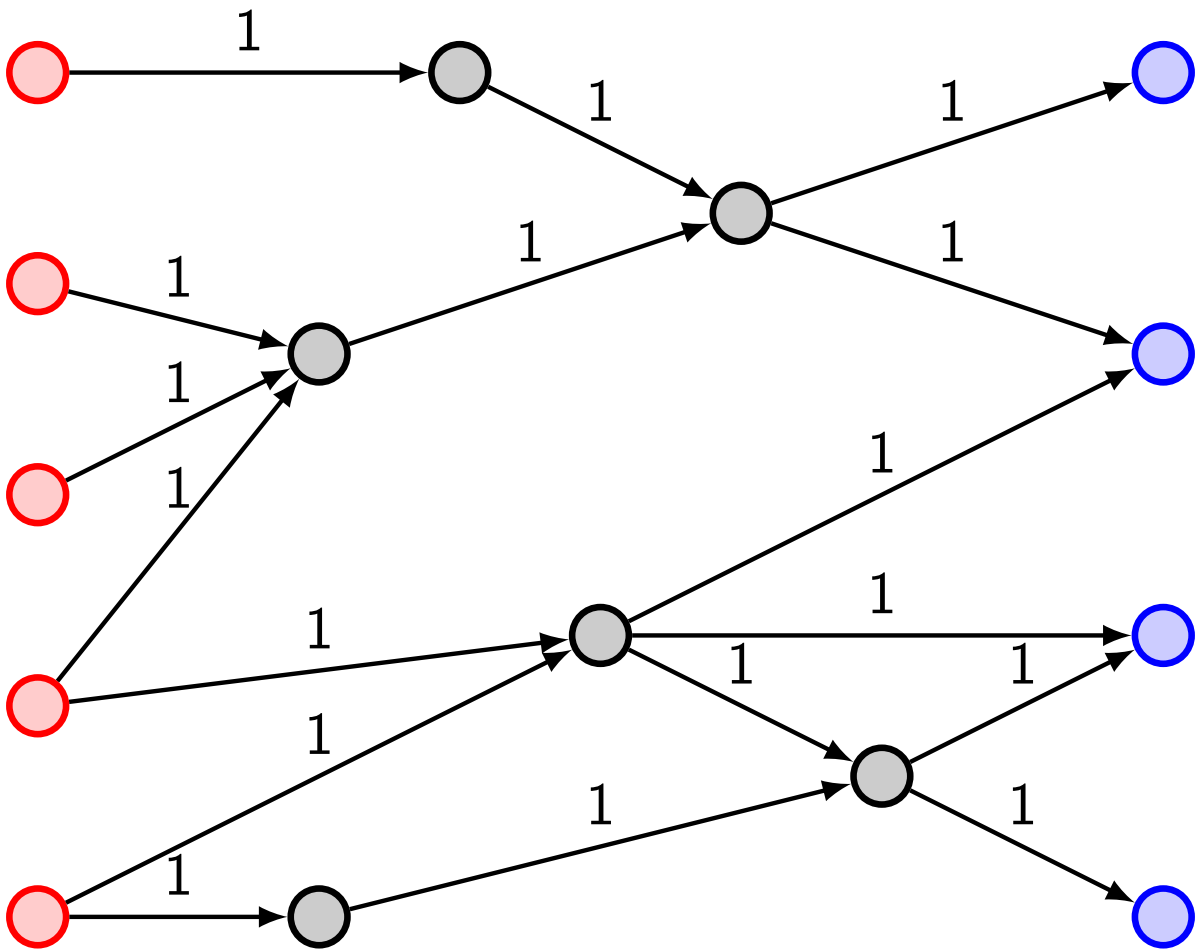
Example problem: Vertex-Disjoint Paths

Construct a flow network with:

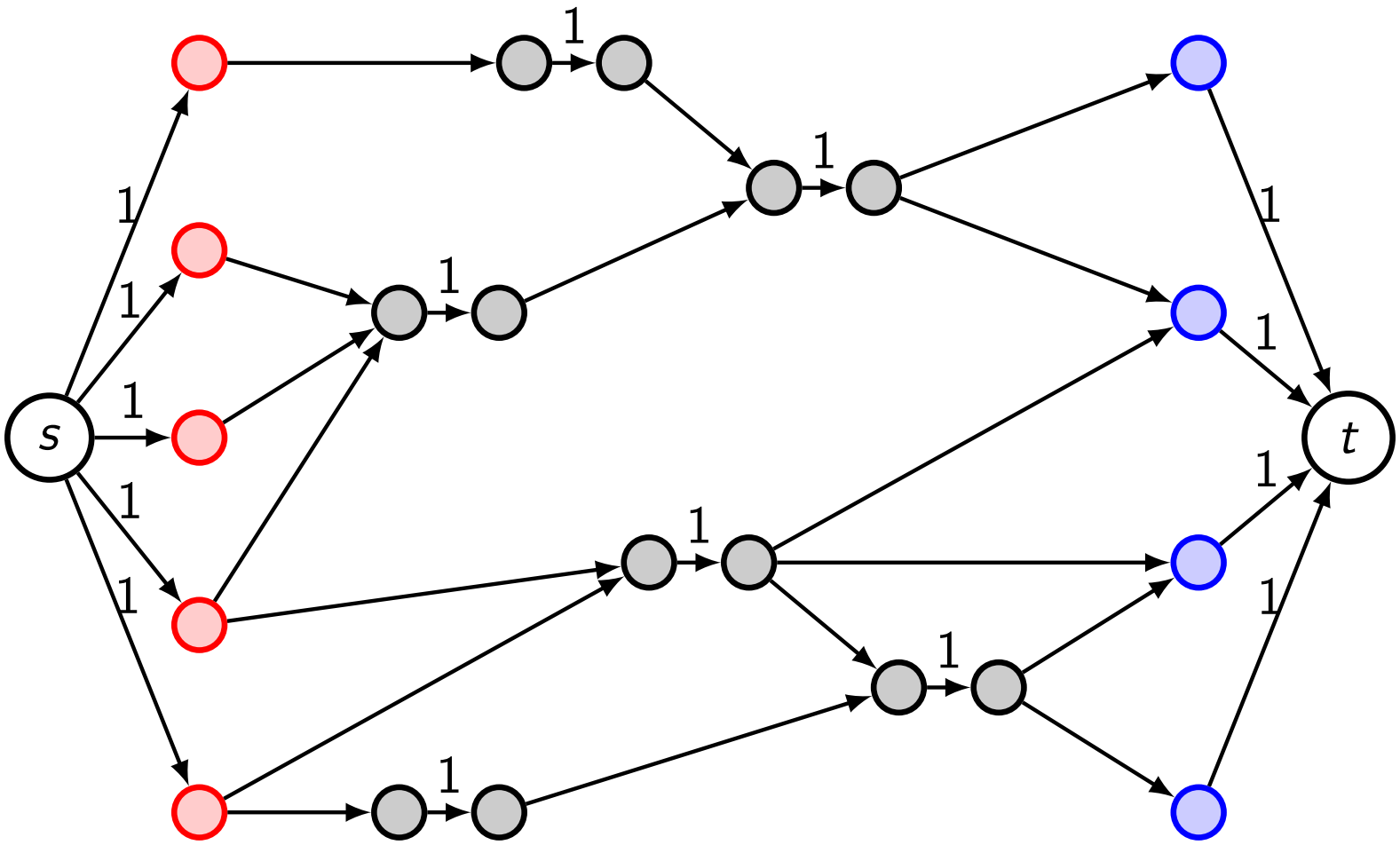
- super-source s joined to each red vertex,
- each blue vertex joined to super-sink t,
- for each black vertex, two vertices v_{in} and v_{out}, joined by an edge,
- each edge of the original graph, with edges from a black vertex drawn from the corresponding out-vertex, and edges to a black vertex drawn to the corresponding in-vertex.

All edges have capacity 1.

Original directed graph with multiple sources (red), multiple sinks (blue), and vertex capacities (of 1 on all black vertices).



Network flow model with a super-source (s), super-sink (t) and edge capacities only.



Example problem: Vertex-Disjoint Paths

Question. What does a flow in this network represent?

- Each edge either has flow 0 or flow 1.
- We interpret each edge with flow 1 as contributing to one of the red-blue paths.
- Flow conservation ensures that each unit of flow travels from s to a red vertex, then to zero or more black vertices, before arriving at a blue vertex and terminating at t.
- By the capacity constraint, each red vertex receives at most one unit of flow from s, so flow conservation ensures that at most one edge from that vertex is flowed, i.e. we use this red vertex in at most one path.
- Similarly, each blue vertex is used in at most one path.
- Likewise, each in-vertex and out-vertex pair contributes to at most one path.
- So our flow corresponds to a set of vertex-disjoint red-blue paths.

Question. What is the running time to find the maximum flow?

- To maximise the number of paths, we find the maximum flow using the Edmonds-Karp algorithm.
- There are at most $2n$ vertices and exactly $n + m$ edges, so the time complexity is

$$O(|V| \cdot |E|^2) = O(n(n+m)^2).$$

- Since the value of any flow is constrained by the total capacities from s and to t, which in this case are r and b respectively, we can achieve a tighter bound of

$$O(|E| \cdot |f|) = O((n+m)\min(r,b)) = O(n(n+m)).$$

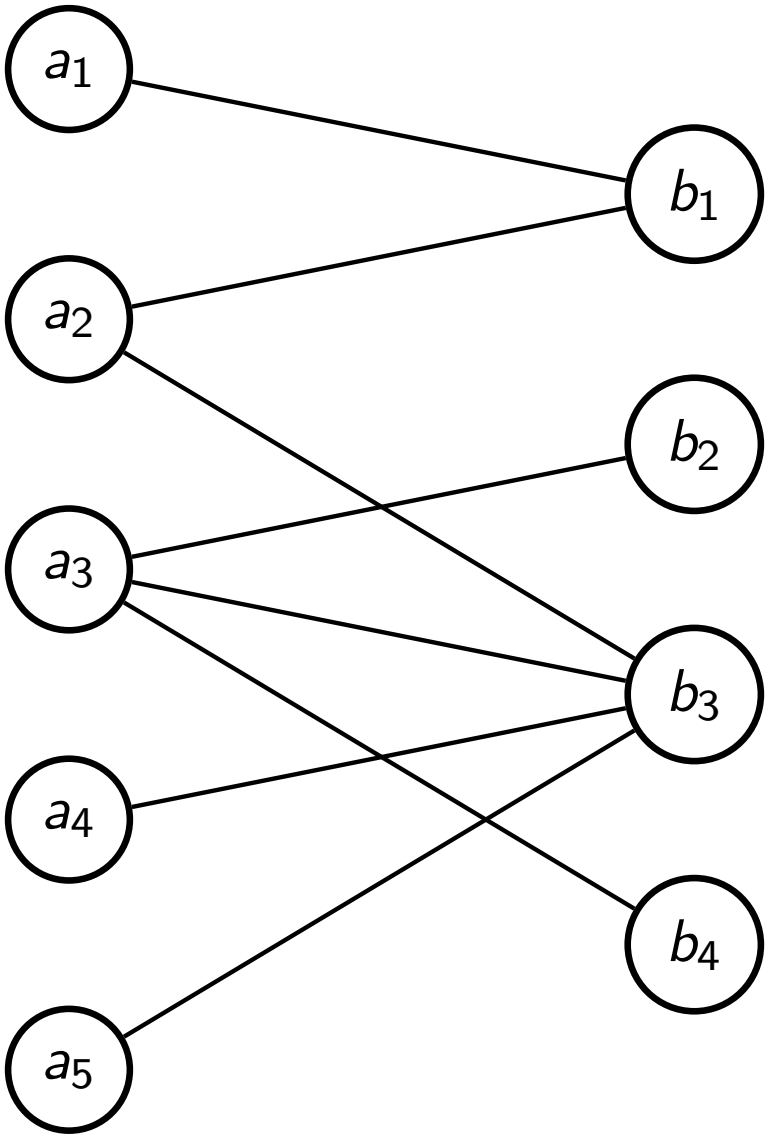
Definition

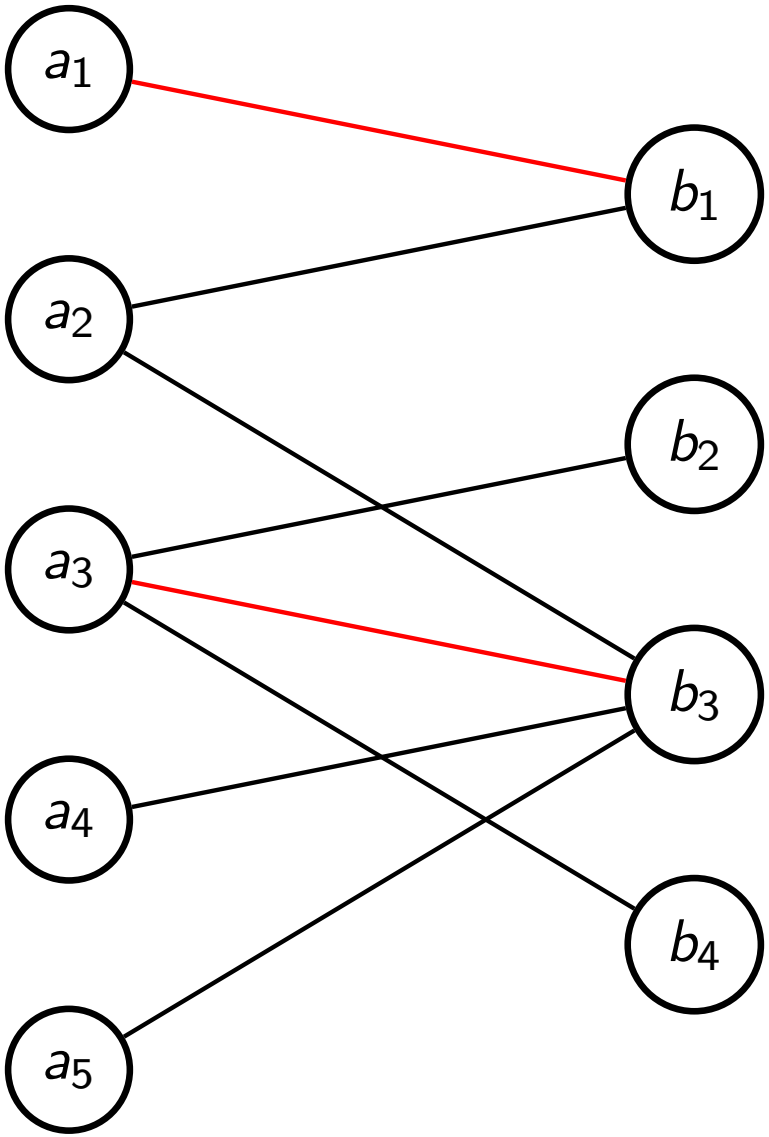
A graph $G = (V, E)$ is said to be <u>bipartite</u> if its vertices can be divided into two disjoint sets A and B such that every edge $e \in E$ has one endpoint in the set A and the other in the set B.

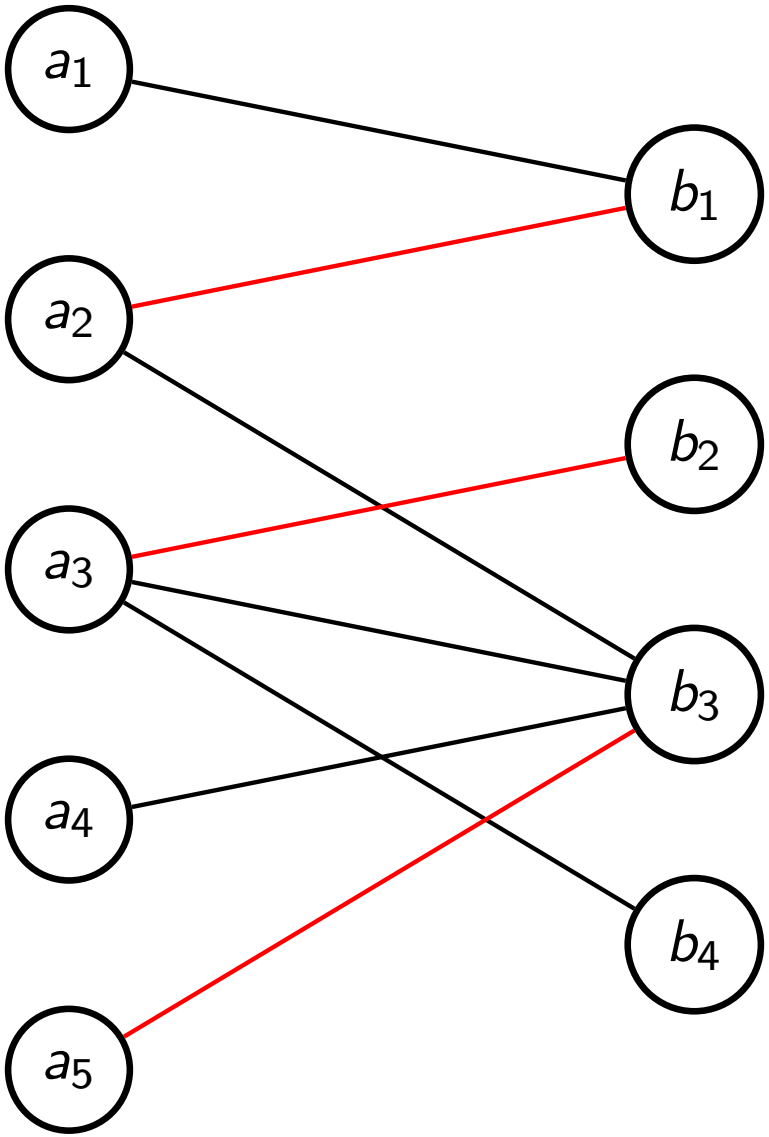
Definition

A <u>matching</u> in a graph $G = (V, E)$ is a subset $M \subseteq E$ such that each vertex of the graph belongs to at most one edge in M.
A <u>maximum matching</u> in G is a matching containing the largest possible number of edges.

Matchings in a Bipartite Graph




a_1
a_2
a_3
a_4
a_5
b_1
b_2
b_3
b_4


a_1
a_2
a_3
a_4
a_5
b_1
b_2
b_3
b_4

Problem

Given a bipartite graph G, find the size (i.e. the number of pairs matched) in a maximum matching.

Question

How can we turn a Maximum Bipartite Matching problem into a Maximum Flow problem?

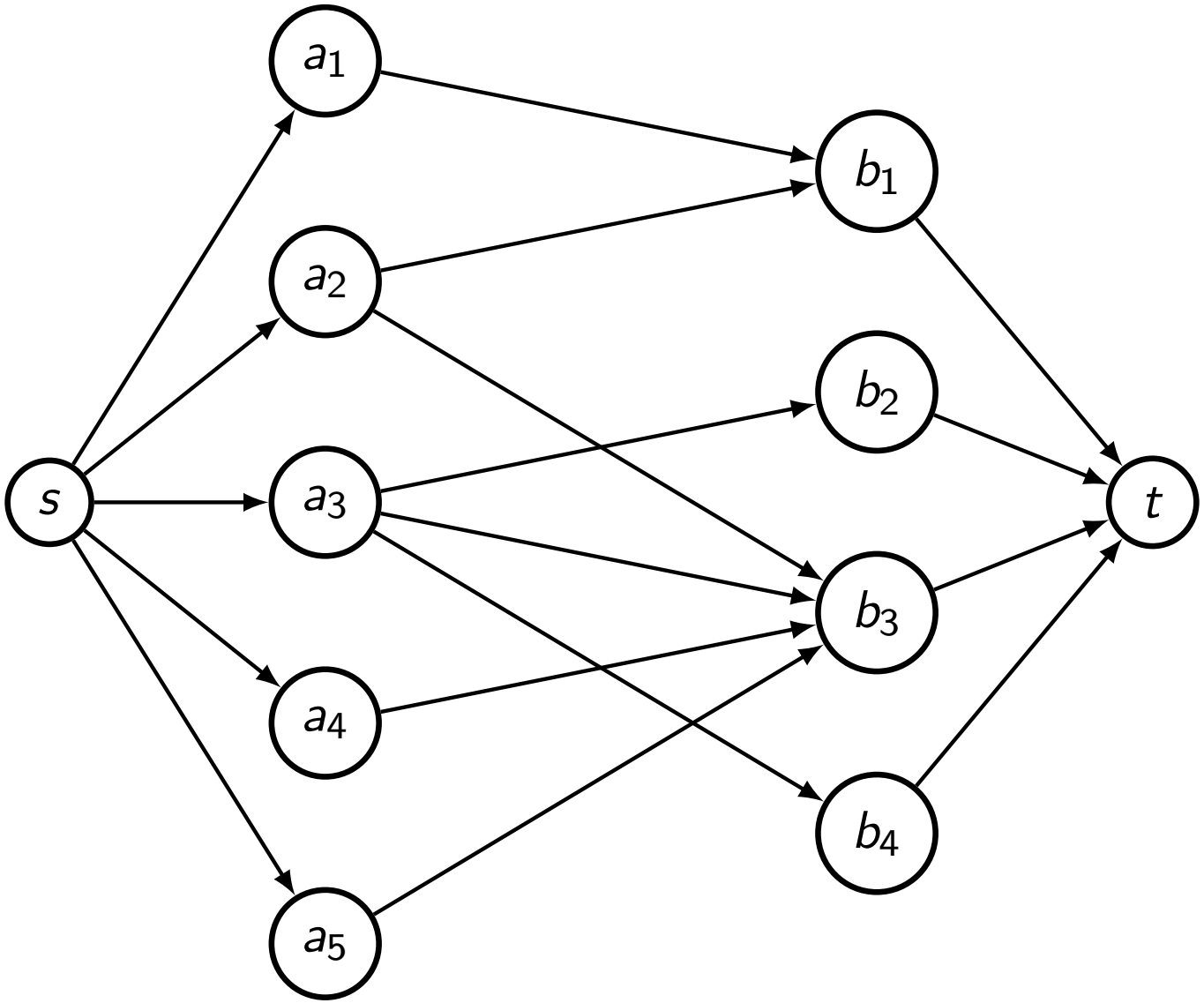
Answer

Create two new vertices, s and t (the source and sink).

Construct an edge from s to each vertex in A, and from each vertex in B to t.

Orient the existing edges from A to B.

Assign capacity 1 to all edges.


a_1
a_2
a_3
a_4
a_5
s
b_1
b_2
b_3
b_4
t

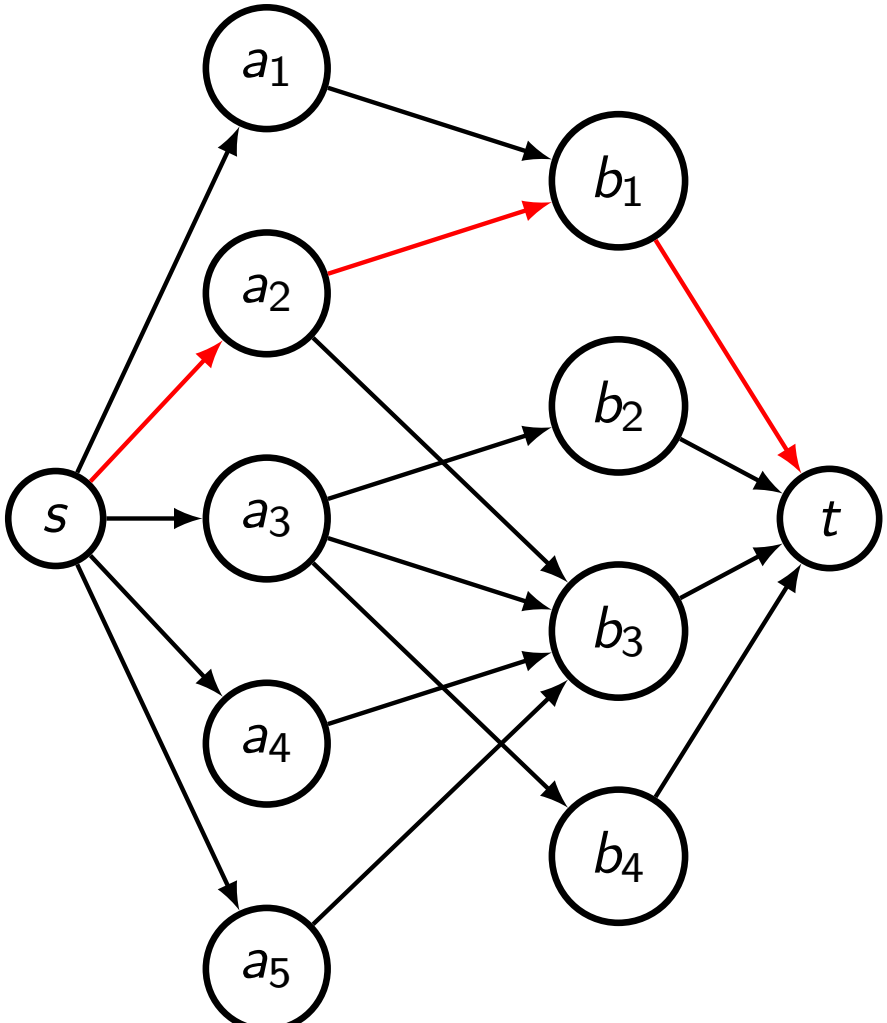
Property

Recall that for each edge $e = (v, w)$ of the flow network, with capacity c and carrying f units of flow, we record two edges in the residual graph:

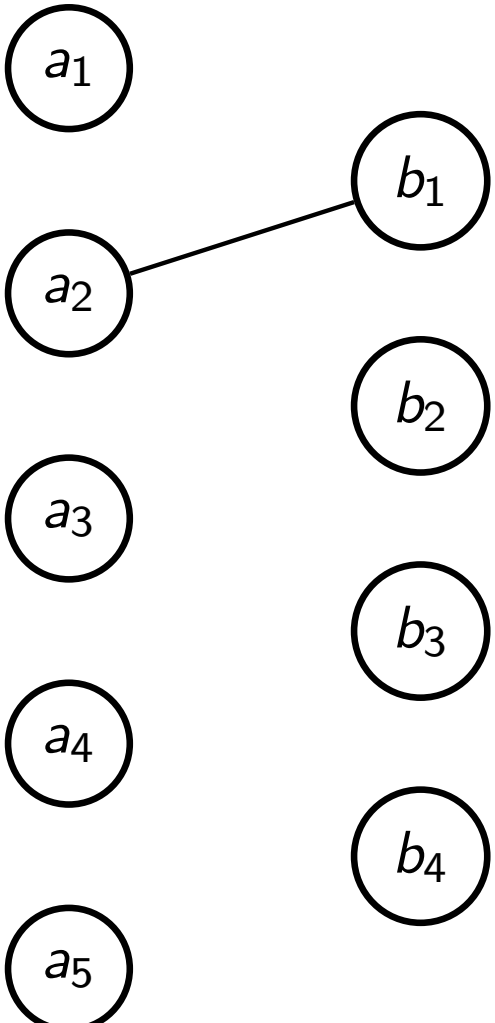
- an edge from v to w with capacity $c - f$
- an edge from w to v with capacity f.

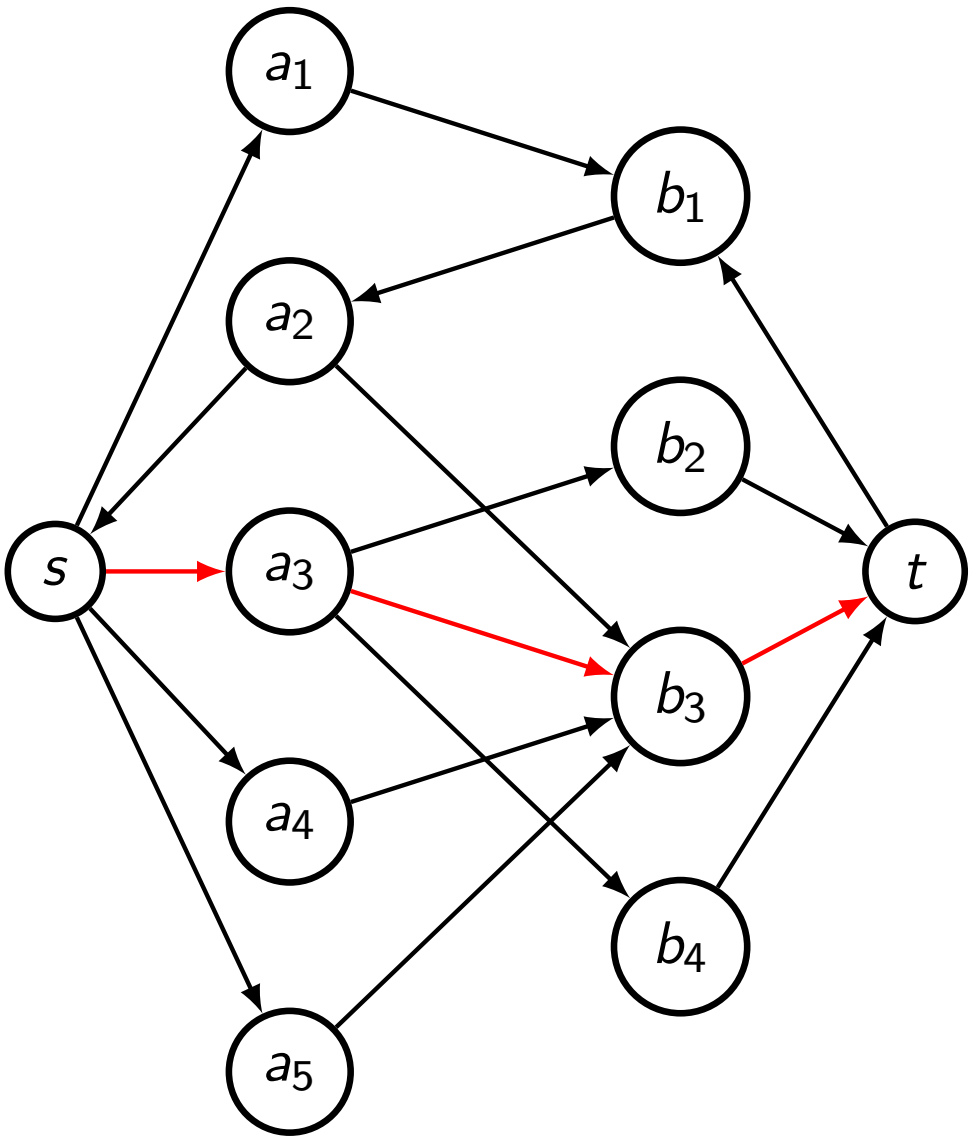
Since all capacities in the flow network are 1, we need only denote the direction of the edge in the residual graph!
As always, the residual graph allows us to correct mistakes and increase the total flow.

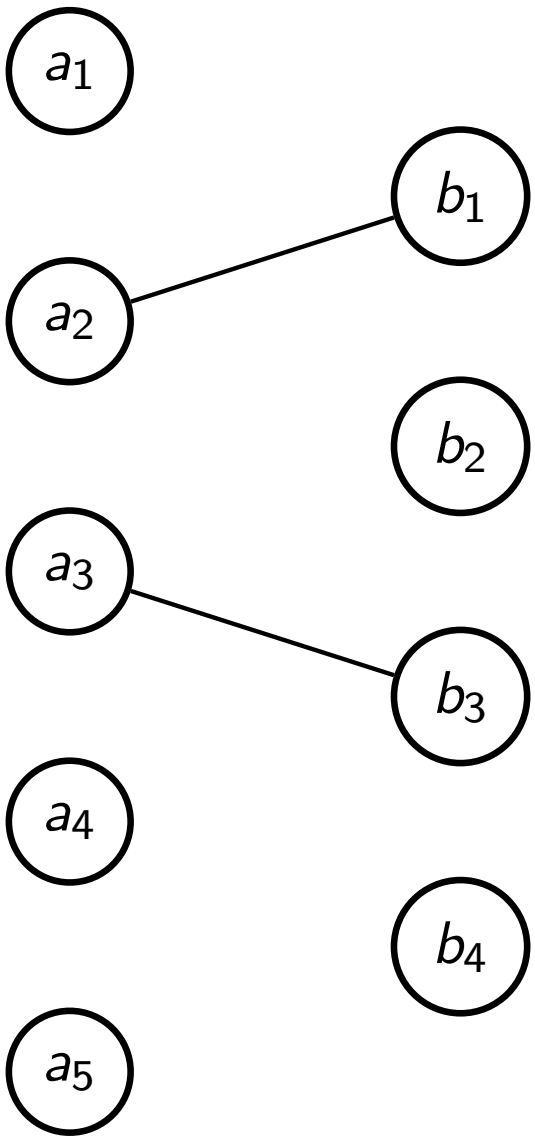
Residual flow network



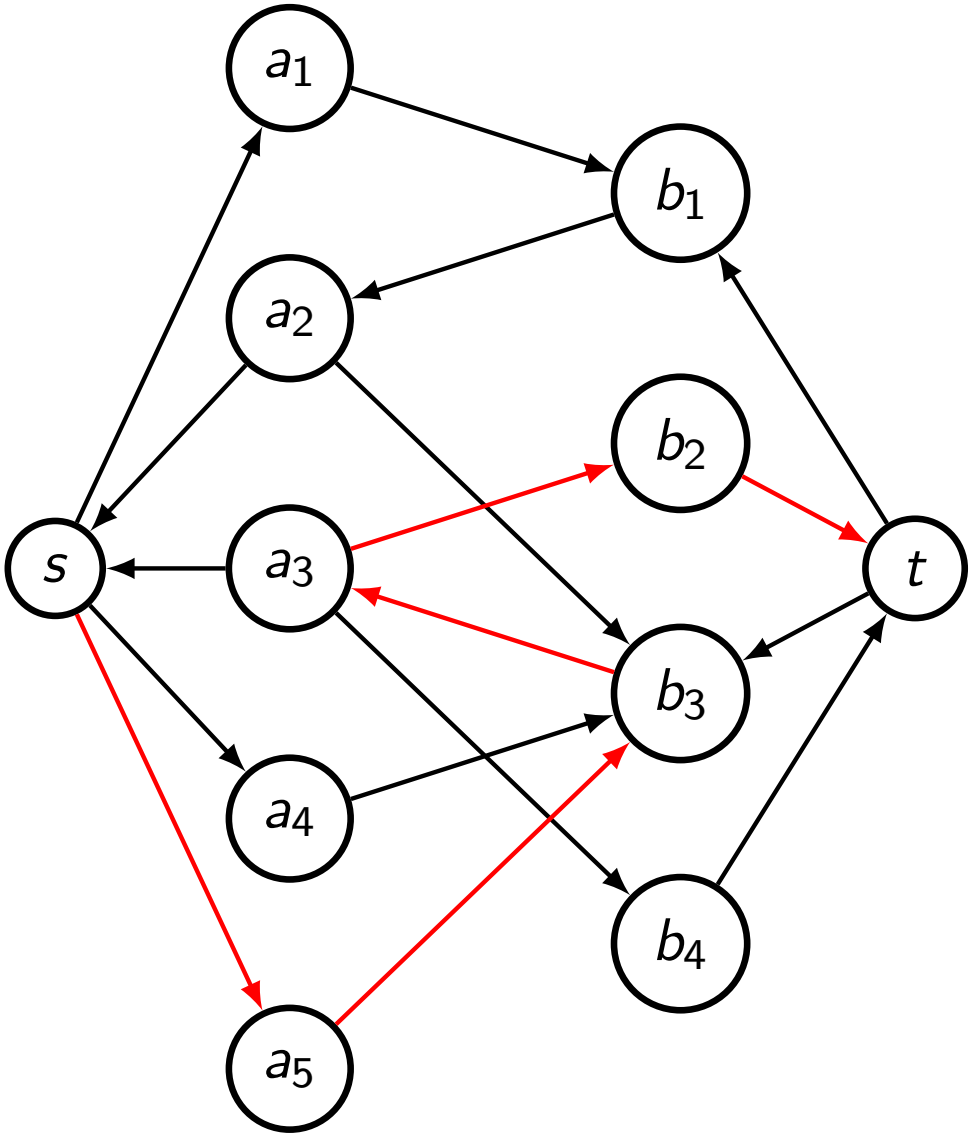
Solution

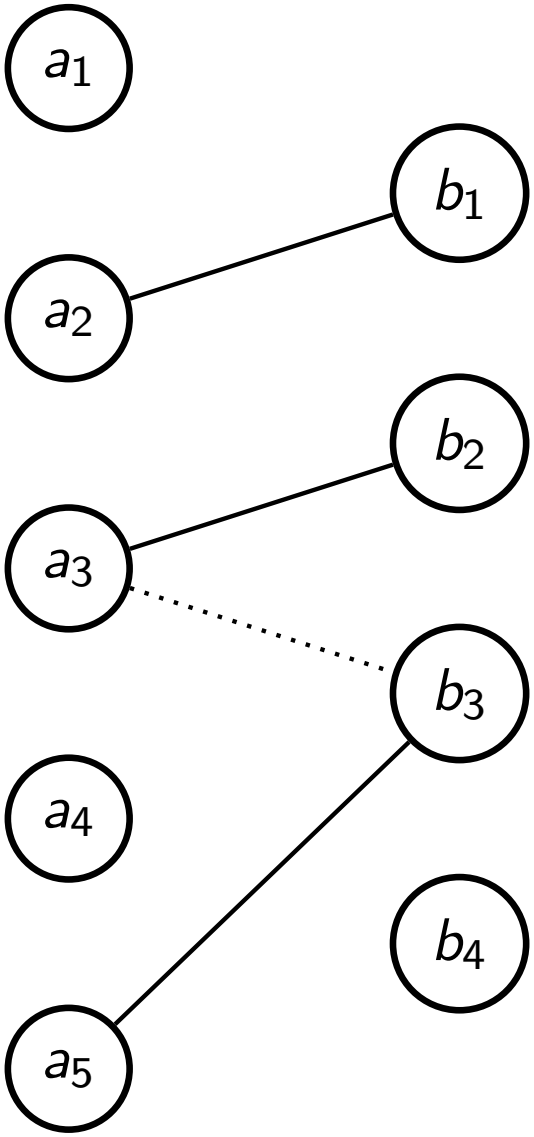



a_1
b_1
a_2
b_2
s
a_3
t
b_3
a_4
b_4
a_5

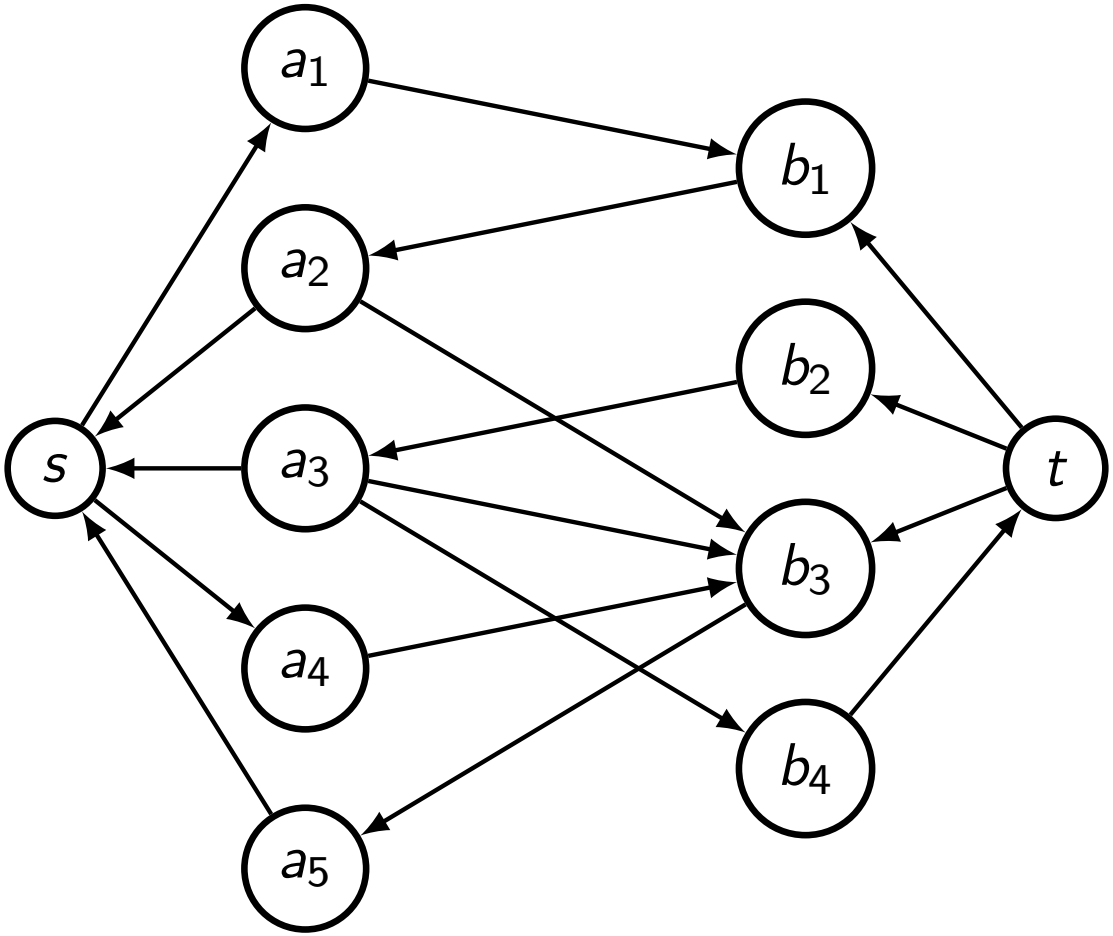

a_1
b_1
a_2
b_2
a_3
b_3
a_4
b_4
a_5

Maximum Bipartite Matching





Final residual flow network



This has no more augmenting paths, so our algorithm terminates.

Example problem: Job Centre

Problem

Instance: You are running a job centre. In your country, there are k recognised qualifications. There are n unemployed people, each holding a subset of the available qualifications. There are also m job openings, each requiring a subset of the qualifications.
Task: Design an algorithm which runs in $O(nm(k + \min(n, m)))$ time and places as many people as possible into jobs for which they are qualified. No worker can take more than one job, and no job can employ more than one worker.

Example problem: Job Centre

- Create an unweighted, undirected graph with vertices $a_1, \ldots, a_n$ and $b_1, \ldots, b_m$, representing the workers and jobs respectively.
- For each $1 \leq i \leq n$ and $1 \leq j \leq m$, place an edge between a_i and b_j if and only if the ith worker holds all qualifications required for the jth job. It is clear that the resulting graph is bipartite.
- Consider a matching in this graph. Each of the selected edges corresponds to the placement of a worker in a job, and we correctly ensure that no worker or job is assigned more than once.
- Therefore, an optimal assignment is a maximum matching in this graph!
- A maximum matching can be found using the Edmonds-Karp algorithm, where the worker-job edges carrying flow comprise the matching.

Example problem: Job Centre

Question. What is the running time to find the maximum bipartite matching?

- For each worker, for each job, we need to iterate through up to k qualifications to determine whether the worker is qualified for the job. This takes $O(nmk)$ time.

- Once again, the tightest bound on the runtime of Edmonds-Karp is of the form $O(|E| \cdot |f|)$, where $|E| \leq nm + n + m$ and $|f| \leq \min(n, m)$.

- Therefore the total time complexity is $O(nm(k + \min(n, m)))$.

Aside: Common Bipartite Graphs

- It is useful to be able to recognize if a problem involves a bipartite graph quickly.
- A sometimes useful characterization: Bipartite graphs are graphs with no odd cycles.
- Also a graph is bipartite iff it can be 2-colored. This allows you to check bipartiteness in $O(|V| + |E|)$ time with a DFS.
- Common examples:
  - Any time you naturally have two different kinds of nodes.
  - On grids, 4-adjacency forms a bipartite graph. Why?
  - The rows and columns of a grid.

A related problem

Given a complete bipartite graph in which both parts have n vertices and each edge has a cost, determine the minimum cost of a perfect matching.

Solution

The Hungarian algorithm solves this problem in $O(n^3)$ time.

It is a special case of weighted matroid intersection that we'll see later.

Example problem: Gusteau's

Problem

Gusteau's restaurant wants to source n bottles of wine from winemakers across the city. Each bottle has to be transported individually.

There are m couriers around the city. The restaurant can hire a courier to collect one bottle of wine, in which case they will travel from their starting position to the winemaker and then to the restaurant, and be paid according to the distance they travel. When a courier reaches the restaurant, you can optionally assign them another collection, and so on.

Given the locations of the winemakers and the starting positions of the couriers (as coordinates relative to the restaurant), design an algorithm which runs in $O((n+m)^3)$ time and determines the minimum cost to transport the wine.

Example problem: Gusteau's

Solution

If couriers couldn't be assigned additional jobs, this would be a minimum cost matching problem. We would make a complete bipartite graph, with couriers on one side and winemakers on the other, and assign edge costs equal to the distance that courier i has to travel to collect wine j and deliver it to the restaurant.

Note that we don't care which couriers are getting re-hired. It's equivalent to having lots of couriers who start at the restaurant. We need to hire at least one of the original couriers, so by 'lots' we mean $n - 1$; enough to pick up <u>almost</u> all the wine.

There may now be more couriers ($m + n - 1$) than wines (n). No problem, just make $m - 1$ wines which each have cost zero for all couriers. Now use the Hungarian algorithm.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Minimum Vertex Cover in a Bipartite Graph

Problem

Given a bipartite graph, determine the size of the smallest vertex cover. Recall that a vertex cover is a subset of vertices S such that every edge is incident to at least one vertex in S.

- Given a maximum matching M, a vertex cover contains at least one endpoint of each edge.
- So, every vertex cover has size at least $|M|$.
- Might we have to take yet more vertices?

Kőnig's Theorem

Theorem

In a bipartite graph, the size of a minimum vertex cover is equal to the size of a maximum matching.

- This is actually equivalent to the max flow min cut theorem.
- It can be proved by constructing a flow network with
  - unit capacity edges from the source and to the sink
  - infinite capacity edges across the bipartition.

Definition

Suppose $G = (V, E)$ is a bipartite graph, with the vertex set V partitioned into independent sets X and Y.

An <u>X-perfect</u> matching is a matching of size $|X|$.

- G has no X-perfect matching if $|X| > |Y|$.
- Particularly interesting if $|X| = |Y|$.
- Naturally important; allows us to pair off <u>every</u> vertex of one side.
- When does G have an X-perfect matching?

Definition

For a subset W of one part X, let the <u>neighbourhood</u> of W (denoted $N(W)$) be the set of vertices adjacent to at least one vertex in W.

In other words, $N(W) = \{y \in Y : \exists w \in W, \{w, y\} \in E\}$.

Theorem

G has an X-perfect matching if and only if for every subset W of X, the neighbourhood of W is at least as large as W, i.e.,

$$|W| \leq |N(W)|.$$

Hall's marriage theorem

- The "only if" direction is easy.
  - If $N(W)$ is smaller than W then there can't be a match for every vertex in W.
- For the 'if' direction, we use the contrapositive.
  - Suppose G has no X-perfect matching, and prove the existence of a set W where $|W| > |N(W)|$.
  - Take a maximum matching, and let $u \in X$ be an unmatched vertex.
  - Now consider all the paths from u which alternate between an edge not in the matching and an edge in the matching. Among the vertices seen in these paths, let W be the subset of X and Z the subset of Y.
  - Two key facts: $|W| > |Z|$ and $Z = N(W)$.

Definition

A graph is <u>k-regular</u> if every vertex has degree k.

Theorem

For $k \geq 1$, a k-regular bipartite graph contains a perfect matching.

Corollary

The edge set of a k-regular bipartite graph can be partitioned into k perfect matchings.

Definition

Let X be a finite set. Then $\leq$ is a partial order on S if the following properties apply

- reflexivity: $x \leq x \; \forall x \in X$.
- antisymmetry: $(x \leq y) \wedge (y \leq x) \implies (x = y)$
- transitivity: $(x \leq y) \wedge (y \leq z) \implies (x \leq z)$.

$(X, \leq)$ is said to be a partially ordered set or poset.

If $x \leq y$ or $y \leq x$, then x and y are said to be comparable; otherwise they are not comparable.

Partial Orders

- Given a directed graph $G = (V, E)$, interpret every directed path from v to w to mean $v \leq w$.

- In what circumstances is $\leq$ a partial order on V?

  - Whenever G is a DAG! (why?)

Definition

In a poset $(X, \leq)$, a <u>chain</u> is a subset of X where every pair of elements is comparable.

Definition

In a poset $(X, \leq)$, a <u>chain decomposition</u> is a partition of X into chains.

Definition

In a poset $(X, \leq)$, an <u>antichain</u> is a subset of X where every pair of elements is <u>not comparable</u>.

Theorem

In a poset, the size of the largest antichain is the smallest number of chains in a chain decomposition.

Dilworth's Theorem: Proof Exercise

- Represent every $x \in X$ by two vertices: x^- and x^+.
- Add the following edges of capacity 1:
  - join the source s to all the 'minus'-vertices,
  - join all the 'plus'-vertices to t, and
  - if x and y are distinct elements of X where $x \leq y$, then add an edge from x^- to y^+.
- Show: there is a decomposition into $|X| - |f|$ chains, where $|f|$ is the maximum flow.
- Show: for any cut (S, T), the set of vertices x such that

$$x^- \in S \text{ and } x^+ \in T$$

  is an antichain of size $|X| - c(S, T)$.
- Conclude the proof of Dilworth's theorem by using the max flow min cut theorem.

Minimum Path Cover in a DAG

Problem

Given a DAG, how many paths must be constructed to cover all vertices?

- Construct a bipartite graph as in the proof of Dilworth's theorem.
- Initially, we have $|V|$ paths each of one vertex, but each edge placed will join two paths and hence reduce the number of paths by one.
- Taking as many edges as possible, we find that the minimum path cover is of size $|V|$ less the size of a maximum matching in this bipartite graph.

Theorem

Let $G = (V, E)$ be a directed, unweighted graph, and let $u, v \in V$ be two non-adjacent vertices.

The number of edge-disjoint paths from u to v equals the size of the smallest (u, v)-separating set of edges, i.e., the fewest edges required to disconnect u from v.

Menger's Theorem: Proof Outline

- Turn G into a flow network with source $s = u$, sink $t = v$, and unit capacities.
- No two paths from s to t can share an edge, so the number of edge-disjoint paths equals the max flow.
- The capacity of a cut is the number of edges crossing it, so the size of the smallest (u, v)-separating set of edges equals the min cut.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Min-cost max-flow

- For each edge e, we also have a cost c_e in our flow network. This is a cost per unit of flow. The cost to send flow f_e through an edge with cost c_e is $c_e \cdot f_e$.
- The minimum-cost maximum-flow problem is to find a maximum flow with smallest possible cost.
- If an edge $v \to w$ has cost c_e, then in the residual graph the edge $w \to v$ will have cost $-c_e$ to 'undo' the effect of sending flow.

Algorithm Outline [no negative cost cycles]

Find the cheapest path from s to t in the residual graph, which will be a min-cost augmenting path. Send flow along this path, adjust the residual graph and repeat until there are no more augmenting paths.

- This generalises Edmonds-Karp, which sends flow along the shortest unweighted path.
- It can be shown that if the original network has no negative cost cycle, then the residual graph will never have one either.
- When this assumption holds, we will find this path using the $O(|V||E|)$ Bellman-Ford algorithm, since Dijkstra's algorithm can't handle negative costs.
- Each path might only send one unit of flow, so the time complexity is $O(|V||E||f|)$.

Algorithm Outline [cycle cancelling]

Find any maximum flow, by ignoring the costs. Then, push flow around any negative cost cycle in the residual graph; this reduces cost without reducing flow.

- This is the cycle cancelling algorithm, and it also works when the cost graph has negative cycles.
- Negative cycles are found using the Bellman-Ford algorithm in $O(|V||E|)$ time.
- Each augmenting cycle decreases the cost by at least 1, so there are at most $|E|UC$ augmenting cycles (where U is the largest of the capacities and C is the largest of the costs, in absolute value).
- The time complexity is $O(|V||E|^2UC)$.

Example problem: Binary Tree on Plane

Problem

You are given n distinct pairs of coordinates, representing points in the plane. Each coordinate is an integer of absolute value at most M.

You want to make a rooted binary tree where:

- each point is a vertex
- each edge points downwards (i.e. any parent has greater y-coordinate than its children), and
- the total length of all the edges (in the sense of Manhattan distance) is minimised.

Design an algorithm which runs in $O(n^5 M)$ time and reports whether the construction is possible, and if so the minimum total length of edges.

Example problem: Binary Tree on Plane

- If it was a general tree, we would greedily pair each vertex to the closest possible parent.
- Min cost max flow allows us to enforce that each vertex has at most two children.

Example problem: Binary Tree on Plane

- Construct:
  - a source s and a sink t,
  - vertices v_p and v_c for v being a parent and a child respectively,
  - edges from s to each v_p of capacity 2 and no cost,
  - edges from each v_c to t of capacity 1 and no cost, and
  - edges from each v_p to each w_c (of lesser y-coordinate) of capacity 1 and cost equal to the distance.
- Constructing the tree is possible iff the max flow is $n-1$. The min cost gives the min total edge length.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Recall: Matroid definition

Definition

A matroid $(S, \mathcal{J})$ consists of a ground set S and a collection $\mathcal{J}$ of subsets of S (called the independent sets) such that:

- **Non-empty:** $\emptyset \in \mathcal{J}$.
- **Hereditary:** If $X \in \mathcal{J}$ and $Y \subseteq X$, then $Y \in \mathcal{J}$.
- **Exchange:** If $X, Y \in \mathcal{J}$ and $|X| > |Y|$, then there exists an $x \in X \setminus Y$ such that $Y \cup \{x\} \in \mathcal{J}$.

- A basis is a maximal independent set, i.e., a basis is an independent set that is not a subset of any other independent set.

- By the exchange property, all bases have the same size.

Definition

For a matroid $(S, \mathcal{J})$, the rank function $r : 2^S \to \mathbb{N}$ is defined as

$$r(X) = \max\{|I| : I \subseteq X, I \in \mathcal{J}\}.$$

It gives, for any subset $X \subseteq S$, the size of the largest independent set restricted to the elements in X.

Definition

The matroid intersection problem is: given two matroids $(S, \mathcal{J}_1)$ and $(S, \mathcal{J}_2)$ on the same ground set S, find the size of the largest set $I \subseteq S$ such that $I \in \mathcal{J}_1$ and $I \in \mathcal{J}_2$.

Consider any independent set I common to both matroids and any subset $X \subseteq S$ of the ground set. Observe that

$$|I| = |I \cap X| + |I \cap (S \setminus X)| \leq r_1(X) + r_2(S \setminus X)$$

where r_1 and r_2 are the rank functions of the two matroids. In particular, this holds for a largest common independent set and a subset X that minimises the sum of ranks:

$$\max_{I \in \mathcal{J}_1 \cap \mathcal{J}_2} |I| \;\leq\; \min_{X \subseteq S} (r_1(X) + r_2(S \setminus X))$$

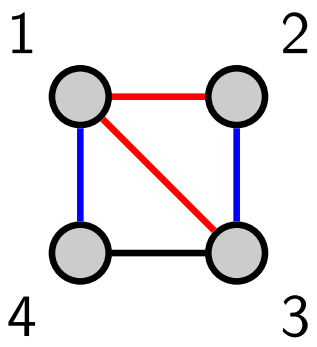
Theorem

For any two matroids $(S, \mathcal{J}_1)$ and $(S, \mathcal{J}_2)$ on the same ground set S with rank functions r_1 and r_2,

$$\max_{I \in \mathcal{J}_1 \cap \mathcal{J}_2} |I| = \min_{X \subseteq S} (r_1(X) + r_2(S \setminus X))$$

We'll give an algorithm that finds a common independent set of this size, which proves the theorem.

Example: colourful spanning trees



For this graph $G = (V, E)$, define the matroids

- $M_1 = (E, \mathcal{J}_1)$ where $\mathcal{J}_1$ is the collection of acyclic edge sets
- $M_2 = (E, \mathcal{J}_2)$ where $\mathcal{J}_2$ is the collection of edge sets with at most one edge of each colour

Then, an independent set common to both matroids is an acyclic set of edges that contains no two edges of the same colour.

Exchange graph

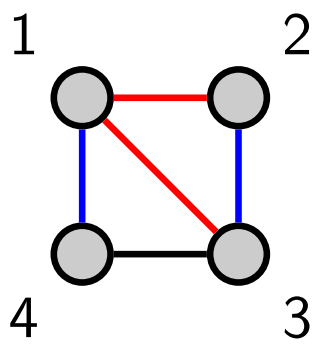
For a set $I \in \mathcal{J}_1 \cap \mathcal{J}_2$, we construct a directed bipartite exchange graph $G(I)$ with bipartition $(I, S \setminus I)$ such that for $x \in I$ and $y \in S \setminus I$,

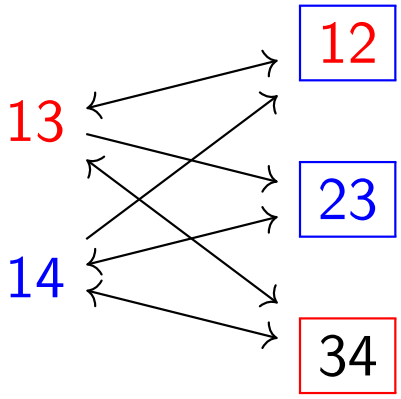
- (x, y) is a directed edge if $(I \setminus \{x\}) \cup \{y\} \in \mathcal{J}_1$ and
- (y, x) is a directed edge if $(I \setminus \{x\}) \cup \{y\} \in \mathcal{J}_2$

The sources are the elements that maintain independence with M_1 when added to I, and the sinks are the elements that maintain independence with M_2 when added to I:

- $s \in S \setminus I$ is a $\boxed{\text{source}}$ if $I \cup \{s\} \in \mathcal{J}_1$ and
- $t \in S \setminus I$ is a $\boxed{\text{sink}}$ if $I \cup \{t\} \in \mathcal{J}_2$

Exchange graph for $I = \{(1,3),(1,4)\}$ with respect to the graphic matroid M_1 and the partition matroid M_2 defined earlier:





We'll want to find a shortest path P from a $\boxed{\text{source}}$ to a $\boxed{\text{sink}}$ in this graph and replace I with $I \Delta P = I \cup P \setminus (I \cap P)$.
Note: if an element $x \in S \setminus I$ is both a source and a sink, this vertex can be added to I directly (it corresponds to a path of length 0): in that case, $I \cup \{x\} \in \mathcal{J}_1 \cap \mathcal{J}_2$.

The matroid intersection algorithm starts with $I = \emptyset$, repeatedly constructs the exchange graph $G(I)$ and finds a shortest path from a source to a sink. If such a path P is found, we I is replaced by $I \Delta P$. If no such path exists, the algorithm terminates and returns I.

Exercise

Show that, at each iteration, I remains an independent set in both matroids.

When the algorithm terminates, let $X \subseteq S$ be the set vertices x such that $G(I)$ has a path from x to a sink.

- X contains all sinks (by definition)

- X contains no source (otherwise, the algorithm would not have terminated)

To prove the matroid intersection theorem, it remains to prove:

Lemma

1. $r_1(X) = |I \cap X|$ and
2. $r_2(S \setminus X) = |I \setminus X|$

1. $r_1(X) = |I \cap X|$
   - Suppose $r_1(X) \neq |I \cap X|$.
   - As $I \cap X$ is independent in M_1, we must have $r_1(X) > |I \cap X|$
   - Thus there exists $y \in X \setminus I$ such that $(I \cap X) \cup \{y\} \in \mathcal{J}_1$
   - But $I \cup \{y\} \notin \mathcal{J}_1$, otherwise X would contain the source y.
   - By the exchange property, as $I \in \mathcal{J}_1$, we can add elements of I to $(I \cap X) \cup \{y\}$ and obtain an independent set $(I \setminus \{x\}) \cup \{y\} \in \mathcal{J}_1$ for some $x \in I \setminus X$.
   - This implies (x, y) is an edge in $G(I)$
   - Since (x, y) is an edge and y has a path to a sink, we have that x has a path to a sink. Therefore, $x \in X$ – a contradiction.

2. $r_2(S \setminus X) = |I \setminus X|$ has a similar proof
   - Suppose $r_2(S \setminus X) \neq |I \setminus X|$.
   - Then, $r_2(S \setminus X) > |I \setminus X|$
   - There is some $y \in (S \setminus X) \setminus I$ such that $(I \cap (S \setminus X)) \cup \{y\} \in \mathcal{J}_2$
   - Again, we can find $x \in I \cap X$ such that $(I \setminus \{x\}) \cup \{y\} \in \mathcal{J}_2$
   - This implies (y, x) is an edge in $G(I)$.
   - Since $x \in X$, we also have that $y \in X$ – a contradiction.

Matroid intersection algorithm

- The algorithm finds a maximum size independent set common to both matroids

- It can be extended to find a common independent set of maximum weight (with weights on the elements of S)

Consider a bipartite graph $G = (U \cup V, E)$ and two matroids

- $M_1 = (E, \mathcal{J}_1)$ where $\mathcal{J}_1$ is the collection of edge sets with at most one edge incident to each vertex in U
- $M_2 = (E, \mathcal{J}_2)$ where $\mathcal{J}_2$ is the collection of edge sets with at most one edge incident to each vertex in V

A matching in G is an independent set common to both matroids. We now obtain Kőnig's theorem as a corollary of the matroid intersection theorem

$$\max_{\text{matching } M} |M| = \min_{X \subseteq E} (r_1(X) + r_2(E \setminus X))$$

as the RHS is the size of a smallest vertex cover. It partitions the edges into those covered by vertices in U and V, respectively.

Table of Contents

1. Flow Networks

2. Solving the Maximum Flow Problem
2.1 Ford-Fulkerson algorithm
2.2 Cuts in a Flow Network
2.3 Max-flow Min-cut (and proof of correctness of FFA)
2.4 Time complexity
2.5 Alternatives to Ford-Fulkerson

3. Applications of Network Flow

4. Combinatorial Theorems

5. Minimum Cost Maximum Flow

6. Matroid Intersection

7. Puzzle

Problem

You are taking two kinds of medicines, A and B, which each come in identical bottles of 30 pills. Pills of medicine A are completely indistinguishable from pills of medicine B. You take one pill of each medicine every day. The pharmacy will only refill your pill bottles every 30 days.

One day, you are down to the last two pills in each bottle when you drop both bottles on the floor, spilling all four pills. Since you cannot tell which are of type A and which are of type B, how can you continue to take one pill of each medicine for the remaining two days?

That's All, Folks!!